



République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Gabès



**Institut Supérieur d'Informatique et de
Multimédia de Gabès
Département Multimédia & Web**

Code mémoire :

MÉMOIRE DE MASTÈRE

Présenté en vue de l'obtention du

**DIPLÔME DE MASTÈRE PROFESSIONNEL EN SÉCURITÉ DES
SYSTÈMES INFORMATIQUES – MPSSI**

Intitulé

**Architecture Zero Trust et Supervision SIEM
pour la Sécurisation des API Bancaires**

Réalisé par

Dalel AKRMI

Sous la direction de

Dr. Akram BELAZI

Soutenu publiquement le septembre 2026 devant le jury composé de :

Président : Dr.

Rapporteur : Dr.

Encadrant : Dr.

Année Universitaire 2025–2026

Institut Supérieur d'Informatique et de Multimédia de Gabès, Campus universitaire – BP 122, 6033 Cité El
Amel 4, Gabès, TUNISIE
Téléphone : (00216) 75 394 229 Fax : (00216) 75 394 309
Email : isimg@isimg.rnu.tn Web : <http://www.isimg.rnu.tn>

Table des matières

Table des figures	3
Liste des tableaux	4
Présentation générale du projet	5
Langue de rédaction	5
1 Introduction générale et état de l'art	6
1.1 Contexte et problématique de la sécurité bancaire ouverte	6
1.2 Standards d'identité et de sécurité financière (FAPI, OAuth 2.0, mTLS)	6
1.3 Le modèle de sécurité Zero Trust (NIST SP 800-207)	7
1.4 De la supervision SIEM passive à la réponse active automatisée (SOAR)	7
2 Spécification du Système et Analyse des Menaces	8
2.1 Périmètre fonctionnel et services bancaires ouverts	8
2.2 Cartographie des acteurs, des actifs sensibles et des frontières de confiance	8
2.3 Exigences fonctionnelles, non-fonctionnelles et sécuritaires	8
2.4 Modélisation des cas d'utilisation	9
2.5 Analyse formelle des menaces (STRIDE)	11
3 Architecture et Conception Technique	13
3.1 Architecture globale et segmentation réseau Zero Trust	13
3.2 Modèle d'identité, d'habilitation et flux d'accès TPP	14
3.3 Modèle de données bancaire sécurisé	18
3.4 Architecture du SIEM et module de réponse active	19
3.5 Modélisation mathématique formelle du score de menace	20
3.6 Module d'analyse assistée par IA et mécanisme de repli heuristique	22
4 Implémentation du Prototype et Traçabilité	24
4.1 Environnement technologique du prototype conteneurisé	24
4.2 Pipeline de filtrage applicatif et dynamique des flux	24
4.3 Journalisation et traçabilité multicouche	26
4.4 Différences et transition vers l'architecture cible de production	29
5 Résultats, Validation et Discussion	31
5.1 Bilan de l'audit de sécurité dynamique automatisé (OWASP ZAP)	31
5.2 Matrice de résilience face au référentiel OWASP API Security Top 10 (2023)	34
5.3 Évaluation expérimentale du mécanisme de défense active	34
5.4 Banc d'essai de performance et impact des mécanismes de sécurité	35
5.5 Interfaces utilisateur et de supervision d'administration	36
5.6 Synthèse de la conformité réglementaire et discussion critique	40
6 Conclusion	41
Perspectives	41
Remerciements	41
ANNEXES	43

Annexe A : Spécifications de Sécurité et Configuration de la Passerelle NGINX	43
Annexe A : Spécifications de Sécurité et Configuration de la Passerelle NGINX . . .	43
A.1 Politique de durcissement SSL/TLS	43
A.2 Filtrage mTLS pour les partenaires TPP et intégrité de l'adresse source	43
A.3 Régulation de débit (Rate Limiting)	43
A.4 En-têtes HTTP de durcissement de sécurité	44
Annexe B : Spécification du Pipeline d'Ingestion SIEM & Dictionnaire de Données	45
Annexe B : Spécification du Pipeline d'Ingestion SIEM & Dictionnaire de Données	45
B.1 Dictionnaire des champs d'événements de sécurité	45
B.2 Modèle d'ingestion Logstash et filtres Grok	45
Annexe C : Protocole d'Audit Dynamique (DAST) & Matrice de Tests OWASP ZAP	46
Annexe C : Protocole d'Audit Dynamique (DAST) & Matrice de Tests OWASP ZAP	46
C.1 Plan de test et matrice de couverture par endpoint	46
C.2 Critères d'acceptation et seuils d'intégration continue (CI/CD)	46
C.3 Synthèse du rapport brut OWASP ZAP (Scan Actif d'API)	47
Annexe D : Algorithme Formel et Automate de la Réponse Active	48
Annexe D : Algorithme Formel et Automate de la Réponse Active	48
D.1 Algorithme d'évaluation du score et de déclenchement du blocage	48
D.2 Machine à états de la gestion d'une adresse IP source	48
Références	49

Table des figures

1	Diagramme des cas d'utilisation du système bancaire sécurisé.	9
2	Architecture de déploiement sécurisée et segmentation réseau Zero Trust. . .	13
3	Séquence d'accès Open Banking via TPP avec authentification mTLS et flux d'autorisation.	14
4	Modèle de données IAM et relations de consentement Open Banking.	16
5	Processus d'authentification forte (2FA) : parcours OTP par courriel (nominal) et TOTP (alternatif).	17
6	Modèle de données global de l'application bancaire.	19
7	Diagramme de classes du module de supervision SIEM et de réponse active. .	20
8	Diagramme d'activité : pipeline de filtrage et de sécurisation d'une requête entrante.	25
9	Flux dynamique de supervision, d'analyse des menaces et d'alerte en temps réel.	28
10	Séquence d'exécution de l'audit de sécurité automatisé avec OWASP ZAP. . .	32
11	Interface de connexion sécurisée proposant l'authentification par mot de passe et l'accès biométrique.	36
12	Interface de vérification du code OTP à 6 chiffres (authentification forte SCA/PSD2).	37
13	Vue détaillée d'un compte bancaire : solde, écritures comptables et masquage des données sensibles.	37
14	Module d'administration des utilisateurs : attribution des rôles et suspension de comptes.	38
15	Interface de gestion du support client avec traçabilité intégrale des échanges.	38
16	Tableau de bord combinant la prise de rendez-vous en agence et la supervision des activités système.	39
17	Tableau de bord SIEM sous Kibana indexant les flux de sécurité NGINX, FastAPI et Keycloak.	39
18	Interface de recherche en temps réel Kibana Discover avec filtres par niveau de sévérité et IP.	40

Liste des tableaux

1	Description détaillée : Interaction globale avec le système bancaire	10
2	Description détaillée : Gestion des consentements Open Banking (TPP)	10
3	Description détaillée : Blocage applicatif automatique d'une adresse IP suspecte	11
4	Modélisation des menaces STRIDE et contre-mesures de sécurité associées. . .	12
5	Description détaillée : Parcours d'accès TPP Open Banking avec mTLS	15
6	Description détaillée : Authentification forte de l'utilisateur (2FA)	18
7	Trajectoire chronologique et calcul formel pas à pas du score T_{decay} par scénario.	21
8	Matrice de confusion de la détection de menace ($N = 1000$ sessions évaluées).	21
9	Description détaillée : Pipeline de filtrage de sécurité de la requête	26
10	Description détaillée : Détection de menace et auto-remédiation SOAR	29
11	Sources, formats et destinations des journaux dans l'architecture.	29
12	Description détaillée : Audit dynamique de sécurité DAST avec OWASP ZAP	33
13	Synthèse des résultats de l'audit de sécurité dynamique applicatif (OWASP ZAP).	33
14	Matrice de couverture et de validation face aux risques OWASP API Security Top 10 (2023).	34
15	Impact comparatif des couches de sécurité sur la latence, la dispersion et le débit (séquentiel $c=1$ et concurrent $c=35$) de l'API bancaire.	35
16	Synthèse de la gouvernance et de la conformité aux standards réglementaires.	40
17	Politique des en-têtes de durcissement HTTP configurés sur la passerelle NGINX.	44
18	Dictionnaire des données de sécurité de l'index Elasticsearch.	45
19	Matrice des endpoints bancaires et vecteurs d'attaque simulés lors de l'audit DAST.	46

Architecture Zero Trust et Supervision SIEM pour la Sécurisation des API Bancaires

Dalel AKRMI¹ Dr. Akram BELAZI²

dalelakremi@isimg.tn akram.belazi@isimg.tn

¹  ORCID  LinkedIn  GitHub

²  ORCID  LinkedIn  GitHub

Département Multimédia et Web, Institut Supérieur d'Informatique et de Multimédia de Gabès (ISIMG), Université de Gabès, Gabès, Tunisie

Résumé

Ce mémoire présente la conception et la validation d'une architecture de sécurité *Zero Trust*, conforme au NIST SP 800-207, pour les API bancaires ouvertes dans le contexte de l'Open Banking et de la directive PSD2. La solution intègre une passerelle NGINX utilisant mTLS, un serveur d'identité Keycloak reposant sur OAuth 2.0 et des jetons signés avec RS256, un backend FastAPI, un frontend Flask ainsi qu'une pile SIEM/SOAR réactive calculant un score dynamique de menace $T_{decay}(x, t)$. L'audit DAST réalisé avec OWASP ZAP n'a révélé aucune vulnérabilité critique dans le périmètre évalué.

Mots clés: Open Banking; Zero Trust; mTLS; SIEM; Keycloak; FastAPI; OWASP; API Security

Acronymes: API: Application Programming Interface, BOLA: Broken Object Level Authorization, DAST: Dynamic Application Security Testing, ELK: Elasticsearch Logstash Kibana, JWT: JSON Web Token, mTLS: Mutual Transport Layer Security, OIDC: OpenID Connect, OTP: One-Time Password, OWASP: Open Worldwide Application Security Project, RBAC: Role-Based Access Control, SIEM: Security Information and Event Management, SOAR: Security Orchestration Automation and Response, TOTP: Time-based One-Time Password, TPP: Third Party Providers, ZAP: Zed Attack Proxy

Présentation générale du projet

Ce projet propose une architecture robuste pour répondre aux défis de l'Open Banking. Il associe une barrière cryptographique au niveau de la couche transport (mTLS) à un mécanisme de défense active (SOAR), capable d'isoler une adresse IP hostile en moins de 30 secondes. L'ensemble a été validé au moyen d'un prototype conteneurisé sous Docker.

Langue de rédaction

Ce mémoire est rédigé en langue française conformément aux normes de l'ISIMG. Les termes techniques et acronymes anglophones consacrés (API, mTLS, JWT, RBAC, SIEM, SOAR, DAST) sont conservés dans leur forme originale et définis dès leur première apparition.

1. Introduction générale et état de l'art

La transformation numérique du secteur bancaire, impulsée par l'adoption de la directive européenne sur les services de paiement (PSD2 / DSP2) Commission Européenne 2015, a conduit les institutions financières à ouvrir leurs systèmes d'information à des prestataires tiers agréés (*Third Party Providers* – TPP). Ces prestataires, qui interviennent comme fournisseurs de services d'information sur les comptes (AISP) ou comme prestataires de services d'initiation de paiement (PISP), interagissent directement avec les interfaces de programmation applicative (API) bancaires. Si cette ouverture favorise l'interopérabilité et l'innovation financière, elle accroît également l'exposition des systèmes bancaires aux cybermenaces.

1.1 Contexte et problématique de la sécurité bancaire ouverte

La sécurisation des API financières se heurte à un changement de paradigme : l'effacement de la frontière périmétrique traditionnelle de l'entreprise. Les modèles de sécurité conventionnels reposent sur une défense de type « château fort » (pare-feux, zones démilitarisées et réseaux locaux implicitement réputés sûrs) et supposent que les flux internes sont dignes de confiance. Dans le contexte de l'Open Banking, où des applications tierces hétérogènes accèdent à des ressources sensibles, cette hypothèse de confiance implicite n'est plus adaptée.

Les dispositifs traditionnels présentent deux écueils majeurs :

- **Les limites des pare-feux applicatifs conventionnels (WAF)** : Les WAF analysent principalement les flux à l'aide de signatures statiques et d'expressions régulières. S'ils sont efficaces contre des attaques classiques, telles que les injections SQL ou les scripts intersites (XSS), ils sont moins adaptés à la détection des abus de logique métier, comme la manipulation d'identifiants dans une attaque BOLA, ou de certaines usurpations de session.
- **La passivité des systèmes SIEM traditionnels** : Les plateformes de gestion des événements et des informations de sécurité (SIEM), telles que la pile ELK ou Splunk, collectent et indexent des volumes considérables de journaux. Toutefois, leur utilisation traditionnelle demeure principalement orientée vers l'analyse après incident ou l'émission d'alertes destinées aux opérateurs du SOC. Face à des attaques automatisées et rapides, le délai d'intervention humaine peut s'avérer insuffisant.

1.2 Standards d'identité et de sécurité financière (FAPI, OAuth 2.0, mTLS)

Pour encadrer techniquement l'ouverture des API, l'industrie financière et les organismes de normalisation ont établi des référentiels exigeants :

- **Financial-grade API (FAPI 1.0 Advanced)** OpenID Foundation 2021 : Spécification émise par l'OpenID Foundation durcissant OAuth 2.0 pour les environnements à haut risque financier. Elle requiert l'authentification mutuelle TLS (mTLS), l'association cryptographique des jetons aux certificats clients (*Certificate-Bound Access Tokens*) et l'usage de PKCE.

- **OAuth 2.0 (RFC 6749) et OpenID Connect (OIDC)** Hardt 2012 : Protocoles de délégation d'autorisation et d'authentification permettant à un utilisateur d'accorder à un TPP un accès restreint à ses comptes sans lui transmettre ses identifiants bancaires.
- **Mutual TLS (RFC 8705)** Campbell et al. 2020 : Authentification mutuelle au niveau de la couche transport (TLS) où le client TPP et la passerelle d'API NGINX présentent chacun un certificat cryptographique validé.
- **JSON Web Token (JWT - RFC 7519)** Jones, Bradley et Sakimura 2015 : Format de jeton compact et auto-contenu, signé numériquement par Keycloak au moyen de l'algorithme asymétrique RS256, permettant une validation d'intégrité et de portée (*scope*) par le backend API sans requête systématique à la base de données.

1.3 Le modèle de sécurité Zero Trust (NIST SP 800-207)

Face à la dissolution du périmètre, notre démarche opérationnalise les sept principes fondamentaux de l'architecture *Zero Trust* définie par le NIST dans la publication spéciale SP 800-207 Rose et al. 2020 :

1. Toutes les sources de données et services de calcul sont considérés comme des ressources discrètes à protéger.
2. L'ensemble des communications réseau est sécurisé, indépendamment de l'emplacement physique ou logique du client.
3. L'accès aux ressources individuelles est concédé sur une base par session, avec le principe du moindre privilège.
4. L'autorisation d'accès est gouvernée par une politique dynamique combinant identité vérifiée, état de l'application et comportement observé.
5. L'organisation surveille en continu l'intégrité et la posture de sécurité de tous les actifs associés.
6. L'authentification et l'autorisation sont strictement exigées et vérifiées avant tout octroi d'accès.
7. L'organisation collecte et analyse en permanence des données contextuelles sur l'état des communications pour affiner dynamiquement sa posture défensive.

1.4 De la supervision SIEM passive à la réponse active automatisée (SOAR)

L'innovation centrale de ce travail repose sur le couplage d'un système de journalisation et d'ingestion (pile ELK) avec un moteur de décision réactif intégré au middleware de l'API. Cette synergie transforme la supervision passive en un automate de réponse active (*Security Orchestration, Automation and Response* – SOAR). Dès la détection d'un comportement hostile caractérisé par un franchissement de seuil de menace, le système applique de façon autonome un blocage applicatif temporaire de l'adresse IP source, tout en notifiant l'analyste de sécurité via un canal temps réel (Server-Sent Events).

2. Spécification du Système et Analyse des Menaces

2.1 Périmètre fonctionnel et services bancaires ouverts

Le démonstrateur bancaire implémente les services fondamentaux exigés dans le cadre d'un système d'Open Banking moderne :

- Consultation des soldes et historique des écritures comptables (AISP).
- Initiation de virements ponctuels et programmés entre comptes (PISP).
- Gestion du carnet de bénéficiaires de confiance.
- Consultation des crédits en cours et suivi budgétaire.
- Gestion du cycle de vie des consentements accordés aux applications tierces (TPP).

2.2 Cartographie des acteurs, des actifs sensibles et des frontières de confiance

L'écosystème comprend trois catégories d'acteurs distincts :

1. **Propriétaire de la ressource (client bancaire)** : Titulaire légitime des comptes, qui interagit avec le système au moyen du portail frontend ou accorde des autorisations précises à des applications tierces.
2. **Client OAuth / TPP (prestataire tiers)** : Entité logicielle externe agréée qui accède aux API après l'établissement d'une connexion mTLS et l'obtention du consentement du client.
3. **Administrateur de la sécurité (analyste SOC)** : Opérateur chargé de surveiller le tableau de bord des alertes, de gérer la liste blanche et d'examiner les incidents de sécurité.

Les actifs sensibles à protéger comprennent les identifiants et les clés cryptographiques stockés dans PostgreSQL et Keycloak, les données financières conservées dans MongoDB, les clés privées associées aux certificats TLS de la banque ainsi que les journaux de sécurité. Les frontières de confiance (*Trust Boundaries*) séparent l'Internet externe, la zone démilitarisée (*dmz-net*), la zone applicative (*backend-tier*), le réseau de persistance (*data-tier*) et la zone de surveillance (*monitoring-tier*).

2.3 Exigences fonctionnelles, non-fonctionnelles et sécuritaires

- **Exigences fonctionnelles** : Exécution conforme des flux financiers, gestion granulaire des consentements et visualisation continue des alertes de sécurité.
- **Exigences sécuritaires** : Authentification forte du client (SCA), authentification mutuelle transport (mTLS) pour les TPP, cloisonnement réseau strict par conteneurs, non-répudiation des opérations via des journaux à altération détectable.
- **Exigences de performance** : Temps de réponse moyen maintenu sous le seuil contractuel de 200 ms (SLA bancaire) et délai de déclenchement du blocage applicatif inférieur à 30 secondes.

2.4 Modélisation des cas d'utilisation

La Figure 1 expose le diagramme des cas d'utilisation du système bancaire sécurisé, en distinguant clairement les opérations réservées aux utilisateurs finaux et administrateurs du parcours d'accès délégué des partenaires TPP.

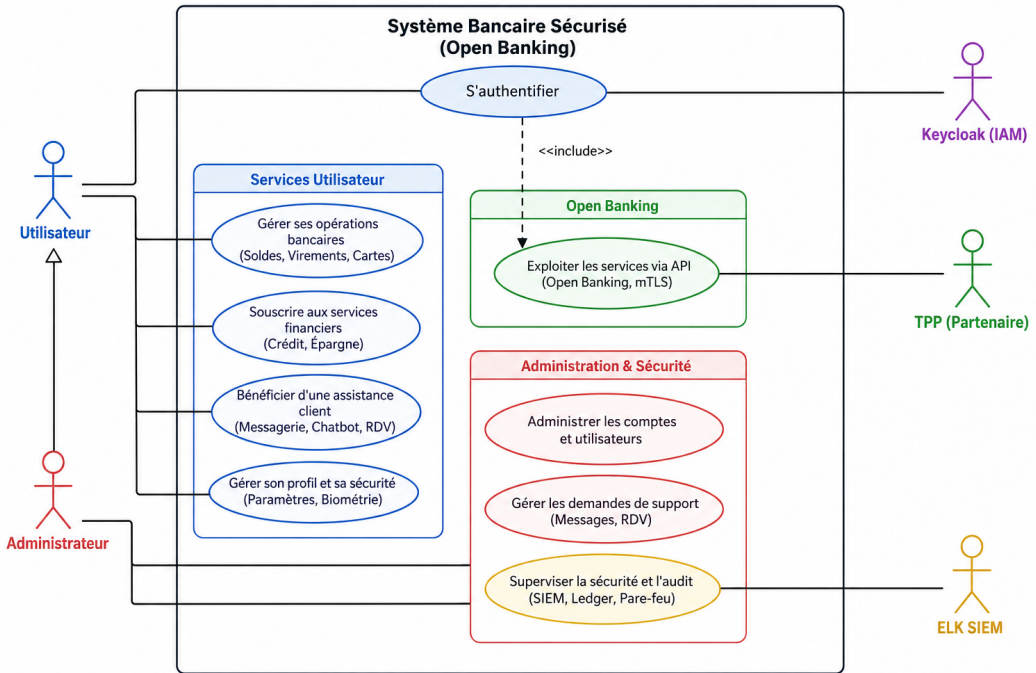


FIGURE 1. Diagramme des cas d'utilisation du système bancaire sécurisé.

Comme l'illustre la Figure 1, l'accès aux services métiers (comptes, virements, consentements) et aux fonctionnalités de supervision (gestion des alertes, blocage IP) est structuré selon des rôles bien définis, évitant les inclusions systématiques superflues et séparant le flux de support administratif du flux bancaire nominal.

Le **Tableau 1** ci-dessous spécifie formellement le cas d'utilisation principal couvrant l'ensemble des interactions entre les acteurs du système (clients, administrateurs, TPP) et l'infrastructure bancaire sécurisée, depuis la réception de la requête jusqu'à son enregistrement dans le journal d'audit.

TABLE 1. Description détaillée : Interaction globale avec le système bancaire

Titre du scénario	Interaction globale avec le système bancaire
Acteur(s)	Client, Administrateur, TPP
Objectif	Permettre l'accès aux services bancaires tout en assurant une surveillance en continu et une isolation hermétique.
Précondition	L'infrastructure conteneurisée est déployée et les services IAM et SIEM sont opérationnels.
Post-condition	L'opération financière ou administrative est exécutée et consignée dans le journal d'audit.
Scénario nominal	1. L'utilisateur ou le TPP transmet sa requête. 2. La passerelle NGINX valide TLS/mTLS. 3. Le backend FastAPI vérifie le jeton JWT et la portée requise. 4. Le service métier traite la demande et enregistre l'événement.
Scénario alternatif	En cas d'échec de vérification ou de comportement hostile, l'événement est capturé par le middleware d'audit, le score de menace est incrémenté et une alerte SSE est transmise au tableau de bord.
Exigences suppl.	Traçabilité totale, conformité aux exigences PSD2/SCA.

Ce tableau met en évidence que chaque interaction, qu'elle soit nominale ou en erreur, produit un enregistrement d'audit structuré, garantissant une traçabilité complète et une réactivité immédiate du système de supervision face à tout comportement anormal.

Le **Tableau 2** ci-dessous détaille le mécanisme de délégation d'accès consenti par le titulaire du compte à une application tierce (TPP), conformément au cadre réglementaire Open Banking imposé par la directive DSP2. Ce scénario est au cœur de l'interopérabilité sécurisée entre la banque et l'écosystème de prestataires.

TABLE 2. Description détaillée : Gestion des consentements Open Banking (TPP)

Titre du scénario	Gestion des consentements Open Banking (TPP)
Acteur(s)	Client bancaire, Application TPP, Serveur Keycloak
Objectif	Accorder une délégation d'accès temporaire et restreinte à un TPP pour la consultation de comptes.
Précondition	Le client possède un compte actif et l'application TPP est authentifiée par certificat mTLS auprès de NGINX.
Post-condition	Le consentement est enregistré et Keycloak délivre un jeton d'accès délimité au TPP.
Scénario nominal	1. Le TPP redirige le navigateur du client vers Keycloak. 2. Le client s'authentifie avec son second facteur. 3. Le client valide les portées demandées. 4. Keycloak crée l'entité Consent et génère le code d'autorisation.
Scénario alternatif	Si le client refuse ou si la portée sollicitée est invalide, Keycloak renvoie un refus d'autorisation au TPP et aucun jeton n'est généré.
Exigences suppl.	Révocabilité immédiate du consentement et principe du moindre privilège.

L'analyse de ce scénario confirme que la gestion des consentements repose sur une architecture de délégation sans stockage de secrets côté TPP : le code d'autorisation est à usage unique et

le jeton JWT émis par Keycloak est strictement délimité aux portées explicitement approuvées par le client, éliminant tout risque de sur-privilege ou d'accès non consenti.

Le **Tableau 3** ci-dessous décrit le déroulement opérationnel complet du mécanisme de détection et de neutralisation autonome d'une adresse IP hostile, illustrant le couplage entre le moteur de score comportemental T_{decay} et le composant d'auto-remédiation SOAR.

TABLE 3. Description détaillée : Blocage applicatif automatique d'une adresse IP suspecte

Titre du scénario	Blocage applicatif automatique d'une adresse IP suspecte
Acteur(s)	Threat Monitor, SecurityState, Administrateur SIEM
Objectif	Isoler de manière autonome une adresse IP source émettant des requêtes malveillantes répétées.
Précondition	Le middleware de sécurité consigne les erreurs HTTP (401, 403, 429) dans le journal d'audit.
Post-condition	L'adresse IP est inscrite dans le registre des adresses bloquées et ses requêtes futures sont rejetées (code 403).
Scénario nominal	1. L'attaquant envoie des requêtes abusives. 2. Le Threat Monitor calcule le score $T_{decay}(x, t)$. 3. Le score dépasse le seuil critique $S = 3.0$. 4. Une alerte CRITICAL est émise via SSE. 5. Un minuteur de 30s s'active. 6. À expiration, SecurityState bloque l'adresse IP.
Scénario alternatif	L'administrateur annule manuellement le blocage sur le tableau de bord avant la fin du minuteur en cas de faux positif identifié.
Exigences suppl.	Découplage asynchrone pour ne pas impacter la latence des flux légitimes.

Ce scénario illustre le principe de défense autonome et proportionnée : le délai de grâce de 30 secondes offert à l'analyste SOC avant l'application définitive du blocage préserve la disponibilité du service pour les utilisateurs légitimes susceptibles de déclencher par erreur le seuil d'alerte, tout en maintenant la réactivité de la réponse aux attaques réelles.

2.5 Analyse formelle des menaces (STRIDE)

L'évaluation méthodique de la résilience du système a été menée au moyen de la méthode de modélisation des menaces STRIDE Kohnfelder et Garg 1999. Le Tableau 4 récapitule les six catégories de menaces identifiées, leur portée technique et les contre-mesures architecturales mises en œuvre dans le cadre de ce projet.

TABLE 4. Modélisation des menaces STRIDE et contre-mesures de sécurité associées.

Menace STRIDE	Définition	Contre-mesure mise en œuvre
Spoofing	Usurpation d'identité d'un client ou d'un TPP	Authentification mutuelle mTLS (certificats X.509 privés) et jetons JWT signés en RS256 par Keycloak.
Tampering	Altération non autorisée des données en transit	Chiffrement systématique TLS 1.3 et validation cryptographique des signatures de jetons.
Repudiation	Répudiation d'une action financière ou d'administration	Traçabilité multicouche par journaux à altération détectable (format JSONL append-only horodaté) et indexation Elasticsearch.
Information Disclosure	Divulgateion de données confidentielles	Cloisonnement des réseaux Docker, validation stricte des schémas Pydantic et masquage du PAN.
Denial of Service	Déni de service et saturation des ressources	Régulation de débit multicouche (NGINX <i>rate limiting</i> et SlowAPI) couplée au blocage applicatif dynamique.
Elevation of Privilege	Élévation non autorisée de privilèges	Contrôle d'accès basé sur les rôles (RBAC) et validation stricte des portées OAuth2 (<i>scopes</i>).

L'examen du Tableau 4 met en évidence que chaque vecteur de menace trouve une contre-mesure technique vérifiable au sein de l'infrastructure, assurant une protection globale conforme aux recommandations de la norme ISO/IEC 27001 ISO/IEC 2022.

3. Architecture et Conception Technique

Notre conception met en pratique le principe fondamental de la **défense en profondeur**, articulé autour de deux paliers complémentaires : la barrière d'accès et d'identité (NGINX, Keycloak, FastAPI, Flask) et le dispositif de surveillance et de réaction (pile ELK, Threat Monitor, SecurityState).

3.1 Architecture globale et segmentation réseau Zero Trust

La Figure 2 présente l'architecture de déploiement sécurisée du système avec isolation réseau stricte.

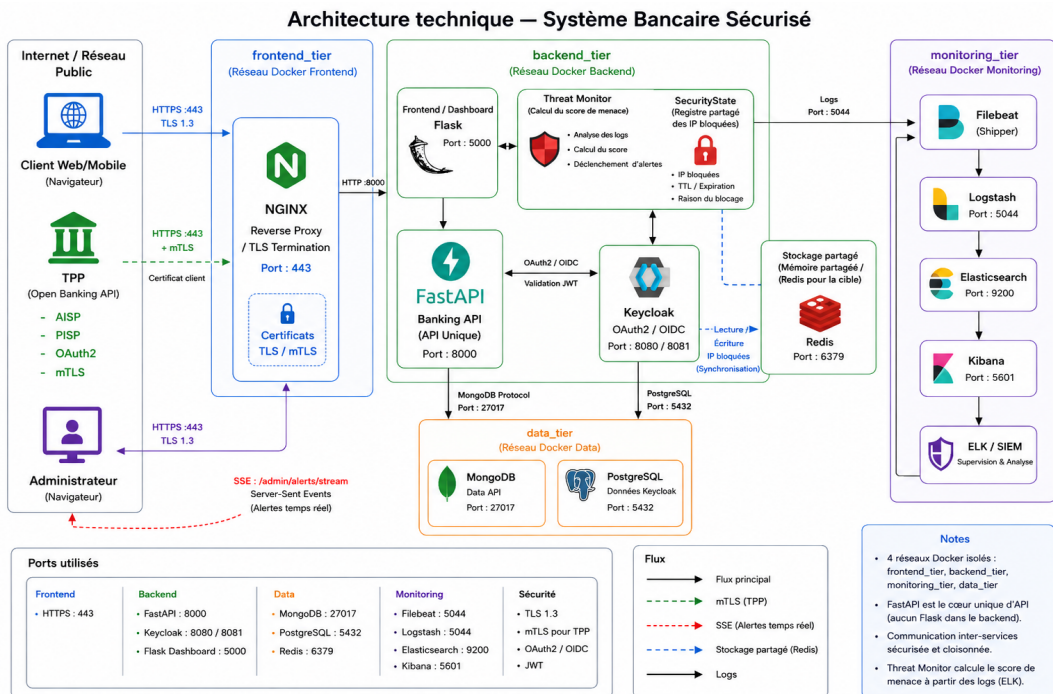


FIGURE 2. Architecture de déploiement sécurisée et segmentation réseau Zero Trust.

Comme le met en évidence la Figure 2, l'infrastructure est cloisonnée en quatre réseaux virtuels isolés :

- **Zone Frontière (DMZ / frontend_tier)** : La passerelle NGINX constitue l'unique point d'entrée exposé sur les ports 80 (redirection HTTPS) et 443. Elle réalise la terminaison TLS, le contrôle strict des certificats mTLS pour les partenaires TPP et applique la première couche de limitation de débit.
- **Zone Applicative (backend_tier)** : Réseau interne abritant le serveur backend FastAPI (port 8000), le serveur frontend/tableau de bord Flask (port 5000), le serveur d'identité Keycloak (ports 8080 / 8081), ainsi que les composants internes de sécurité *ThreatMonitor* (calcul du score de menace), *SecurityState* (registre des adresses IP bloquées avec synchro-

nisation) et le stockage partagé (*Redis* sur le port 6379 pour l'architecture cible). Aucun de ces services n'est accessible directement depuis l'extérieur sans traverser NGINX.

- **Cœur de Données (data_tier)** : Sous-réseau à connectivité interne exclusive (`internal: true`), hébergeant la base MongoDB (port 27017 pour les données financières) et la base PostgreSQL (port 5432 pour les métadonnées Keycloak).
- **Zone de Surveillance (monitoring_tier)** : Réseau dédié à la supervision de sécurité, interconnectant Filebeat (port 5044), Logstash (port 5044), Elasticsearch (port 9200) et Kibana (port 5601).

3.2 Modèle d'identité, d'habilitation et flux d'accès TPP

La passerelle NGINX authentifie les applications tierces (TPP) au moyen de certificats de test émis par une autorité privée (CA interne de la banque). La Figure 3 détaille la séquence d'accès Open Banking d'un TPP mettant en jeu le protocole Authorization Code avec mTLS.

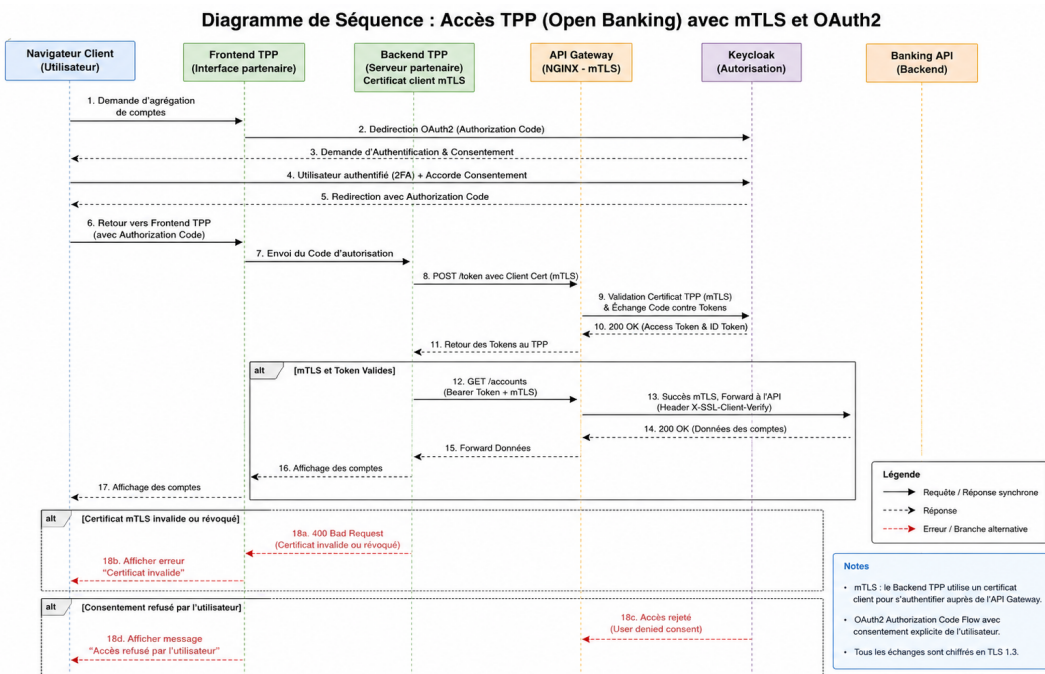


FIGURE 3. Séquence d'accès Open Banking via TPP avec authentification mTLS et flux d'autorisation.

L'analyse de la séquence présentée en Figure 3 montre que la connexion mTLS est validée avant toute communication applicative. La séquence distingue trois entités côté TPP : le navigateur du client final (qui effectue la redirection d'autorisation), le frontend du TPP (interface web) et le backend du TPP (serveur qui échange le code d'autorisation contre le jeton). Ce découplage est essentiel pour la conformité FAPI : seul le backend, qui détient la clé privée mTLS, communique directement avec l'API bancaire. Le TPP redirige le navigateur du client vers Keycloak avec les paramètres `state`, `nonce`, `code_challenge` (PKCE) et les

portées souhaitées (`accounts:read`, `payments:write`). Après authentification forte du client et enregistrement explicite du consentement, Keycloak délivre un code d'autorisation échangé par le backend du TPP contre un jeton JWT RS256. En cas d'échec à l'une des étapes (certificat mTLS invalide, consentement refusé, portée non autorisée ou jeton expiré), la passerelle ou Keycloak retourne un code d'erreur HTTP approprié (400, 401 ou 403) et l'événement est consigné dans le journal d'audit.

Le tableau de scénario ci-après formalise les étapes protocolaires et les règles de sécurité encadrant l'accès des prestataires tiers (TPP).

TABLE 5. Description détaillée : Parcours d'accès TPP Open Banking avec mTLS

Titre du scénario	Parcours d'accès TPP Open Banking avec mTLS
Acteur(s)	Navigateur Client, Backend TPP, NGINX Gateway, Keycloak, Backend FastAPI
Objectif	Délivrer un accès sécurisé aux ressources bancaires à un tiers de confiance dûment authentifié.
Précondition	Le TPP dispose d'une clé privée et d'un certificat émis par la CA privée de la banque.
Post-condition	Le TPP obtient un jeton JWT valide et accède aux données d'information sur les comptes.
Scénario nominal	1. NGINX valide le certificat mTLS du TPP. 2. Le client s'authentifie et accorde son consentement sur Keycloak. 3. Le TPP échange le code d'autorisation contre un JWT. 4. Le TPP consomme l'API FastAPI avec le JWT.
Scénario alternatif	En cas de certificat mTLS révoqué, de portée non consentie ou de jeton expiré, la passerelle ou l'API retourne un code d'erreur (400, 401 ou 403) et notifie le système de journalisation.
Exigences suppl.	Conformité FAPI 1.0 Advanced et validation sans état des jetons.

Ce scénario illustre l'efficacité de la double barrière de contrôle : l'authentification cryptographique mutuelle au niveau transport (mTLS) garantit l'identité du TPP, tandis que le jeton JWT RS256 borne strictement ses privilèges d'accès.

Le modèle de classes du système d'identité et de gestion des accès (IAM) est modélisé dans la Figure 4.

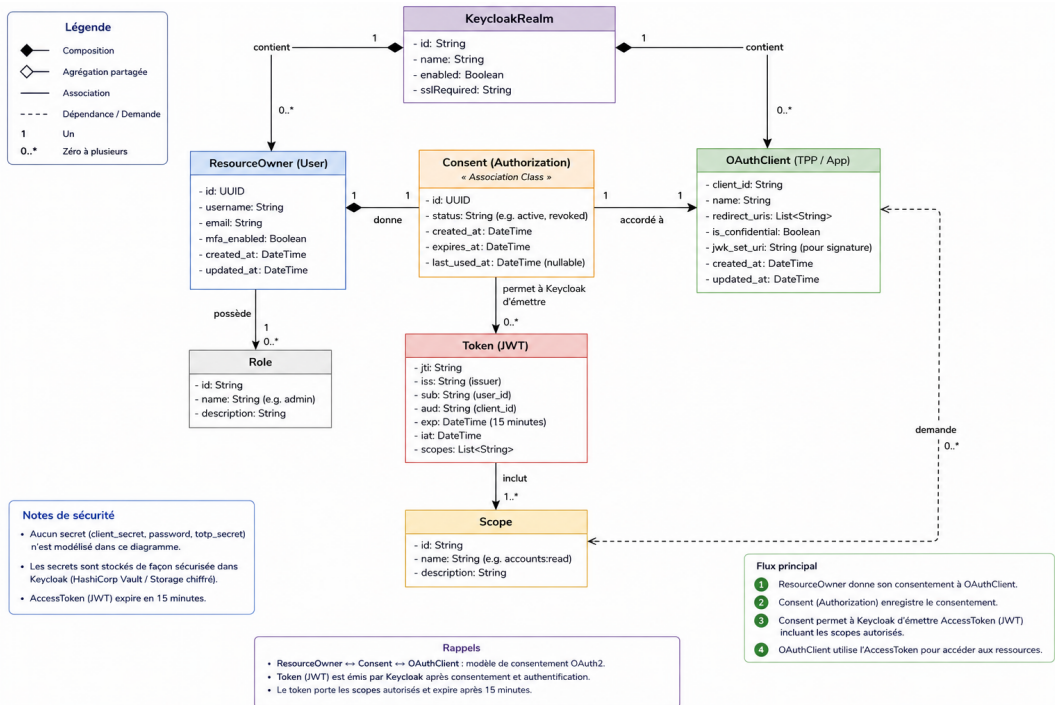


FIGURE 4. Modèle de données IAM et relations de consentement Open Banking.

Comme explicité dans la Figure 4, le modèle dissocie rigoureusement l'identité de l'utilisateur (*ResourceOwner*) de l'application cliente tierce (*OAuthClient*). L'entité *Consent* matérialise l'association entre le propriétaire des données et le client OAuth pour une liste définie de portées (*Scope*), avec date de création et d'expiration. Les secrets sensibles (mots de passe hachés, clés privées, secrets clients) sont confinés au sein du référentiel sécurisé de Keycloak et n'apparaissent pas en clair dans le modèle de domaine métier.

Le processus d'authentification forte à double facteur (2FA) des utilisateurs humains est illustré par la Figure 5.

Diagramme de Séquence : Authentification 2FA (Keycloak + TOTP)

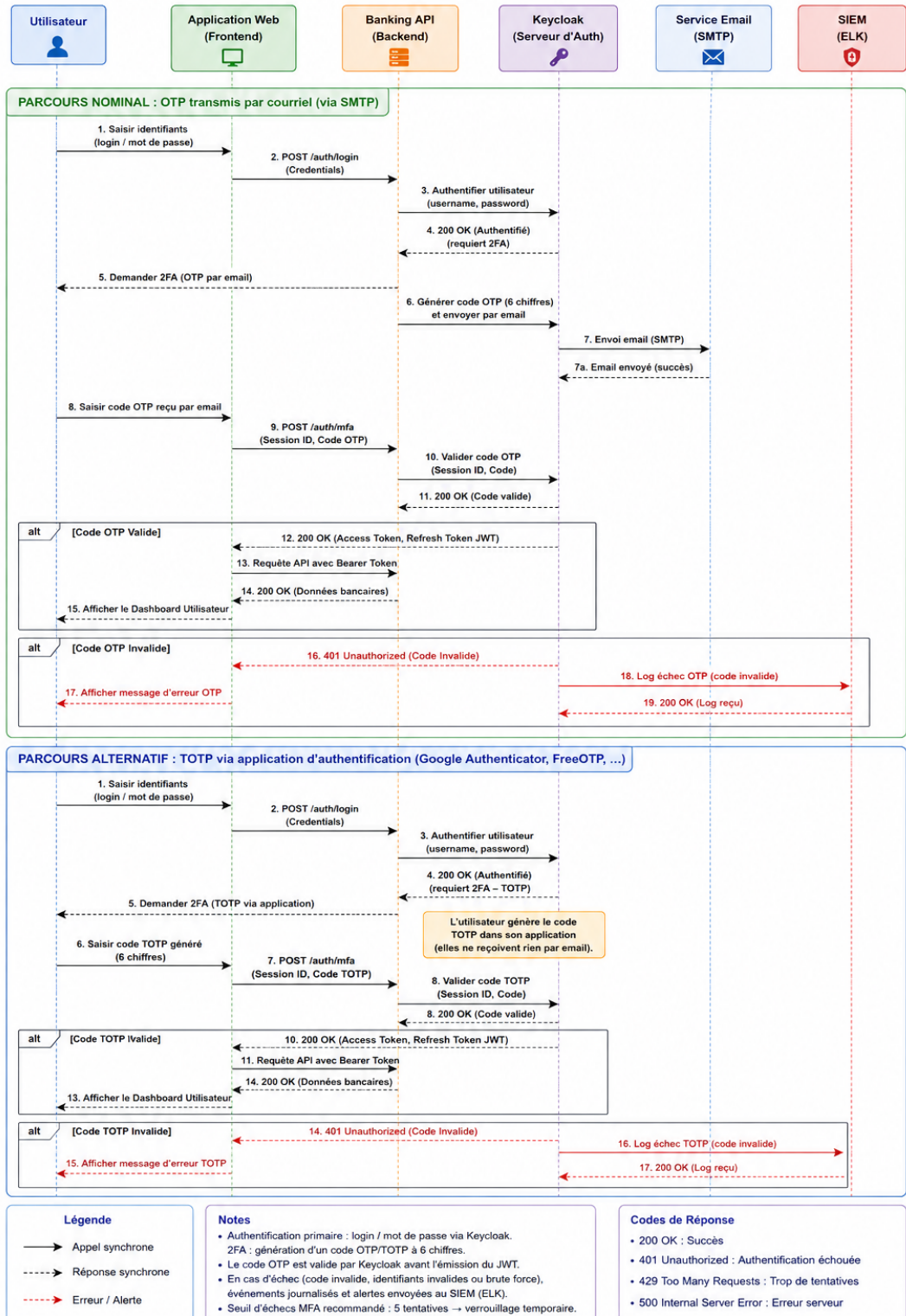


FIGURE 5. Processus d'authentification forte (2FA) : parcours OTP par courriel (nominal) et TOTP (alternatif).

L'enchaînement décrit par la Figure 5 démontre que l'accès utilisateur exige la validation successive de deux facteurs distincts. Le premier facteur est le mot de passe principal (facteur de connaissance), intégralement géré et haché par Keycloak — aucun attribut `password_hash` n'est stocké dans le modèle de domaine métier de l'application. Le second facteur (facteur de possession) est implémenté selon deux parcours exclusifs : **(a) OTP par courriel** (parcours nominal) : Keycloak génère un code à usage unique à 6 chiffres transmis via SMTP au client, valable 5 minutes ; **(b) TOTP** (parcours alternatif) : le client enregistre une application d'authentification (Google Authenticator, FreeOTP) et génère un code RFC 6238 renouvelé toutes les 30 secondes. Les deux options sont configurables individuellement par compte dans Keycloak. Pour l'option d'authentification biométrique proposée au niveau du portail frontend, le gabarit facial est analysé localement avec contrôle de vivacité (*liveness detection*) pour se prémunir des attaques par rejeu, avec bascule systématique vers la procédure nominale (mot de passe + OTP) en cas d'échec.

Le tableau ci-dessous récapitule les caractéristiques formelles du cas d'utilisation d'authentification forte.

TABLE 6. Description détaillée : Authentification forte de l'utilisateur (2FA)

Titre du scénario	Authentification forte de l'utilisateur (2FA)
Acteur(s)	Utilisateur, Frontend Flask, Serveur Keycloak, Service SMTP
Objectif	Valider l'identité du client bancaire conformément aux exigences SCA de la directive PSD2.
Précondition	L'utilisateur possède un compte actif et une adresse électronique vérifiée (parcours OTP) ou une application TOTP enregistrée (parcours alternatif).
Post-condition	L'utilisateur reçoit un jeton d'accès JWT et un jeton de rafraîchissement (<i>Refresh Token</i>).
Scénario nominal	1. L'utilisateur saisit son identifiant et mot de passe (validé par Keycloak). 2a. (Parcours OTP) Le système génère un code OTP transmis par courriel. 2b. (Parcours TOTP) L'utilisateur consulte son application d'authentification. 3. L'utilisateur saisit le code éphémère valide. 4. Les jetons de session sont émis.
Scénario alternatif	En cas de code erroné ou expiré, la tentative est rejetée, l'échec est tracé dans le journal Keycloak et le compteur d'échecs est incrémenté.
Exigences suppl.	Rotation systématique des jetons de rafraîchissement et expiration courte du jeton d'accès (15 minutes).

Cette spécification assure la stricte conformité aux exigences réglementaires SCA (*Strong Customer Authentication*) en éliminant les vulnérabilités de compromission par simple usurpation d'identifiant statique.

3.3 Modèle de données bancaire sécurisé

Le modèle de données de l'application bancaire est représenté sur la Figure 6.

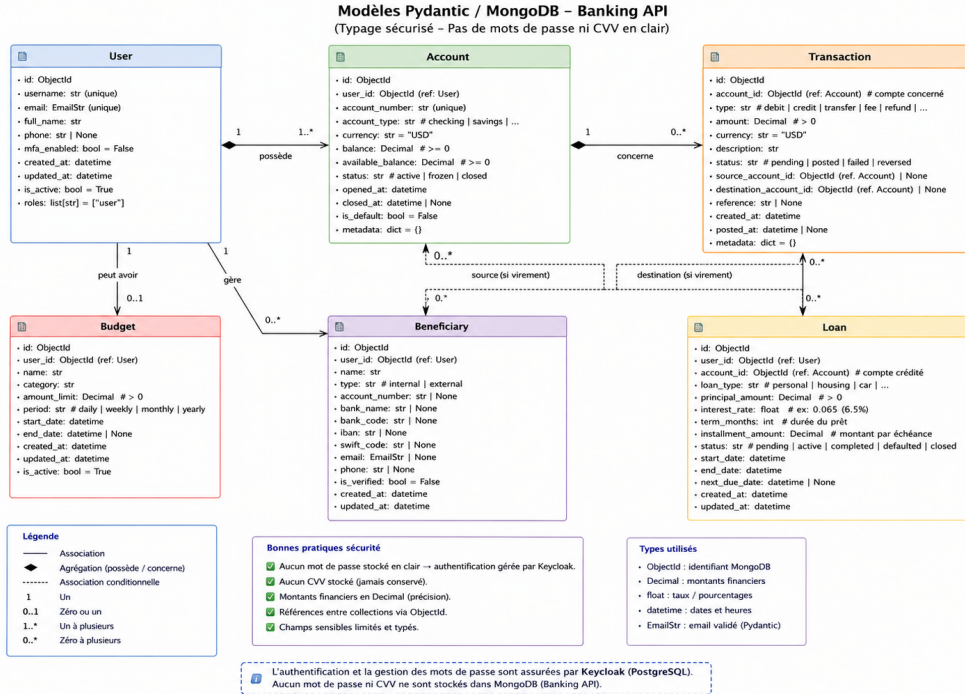


FIGURE 6. Modèle de données global de l'application bancaire.

L'examen de la Figure 6 illustre les règles de conception appliquées pour préserver l'intégrité et la confidentialité des données financières :

- **Précision monétaire (Decimal)** : Les montants des entités *Account*, *Transaction*, *Budget* et *Loan* utilisent le type à virgule fixe `Decimal`, excluant tout risque d'imprécision d'arrondi inhérent aux types flottants standard.
- **Typage strict et validation Pydantic** : Les champs sensibles et les adresses électroniques sont contrôlés à l'ingestion via `EmailStr` et les liaisons inter-collections s'effectuent par identifiants `ObjectId` uniques.
- **Protection des secrets et conformité PCI-DSS** : Aucun mot de passe n'est stocké en clair (seul un condensat haché sous `bcrypt/argon2` est conservé), et les codes de sécurité de cartes (CVV) ne sont jamais persistés en base de données.
- **Modélisation complète du domaine bancaire** : Les flux monétaires (*Transaction*), les plafonds de dépenses (*Budget*), le carnet de tiers de confiance (*Beneficiary*) et les échéances de crédit (*Loan*) sont structurés selon des schémas explicites et modulaires.

3.4 Architecture du SIEM et module de réponse active

La Figure 7 expose le diagramme de classes des composants de journalisation, d'analyse de menace et de réponse active.

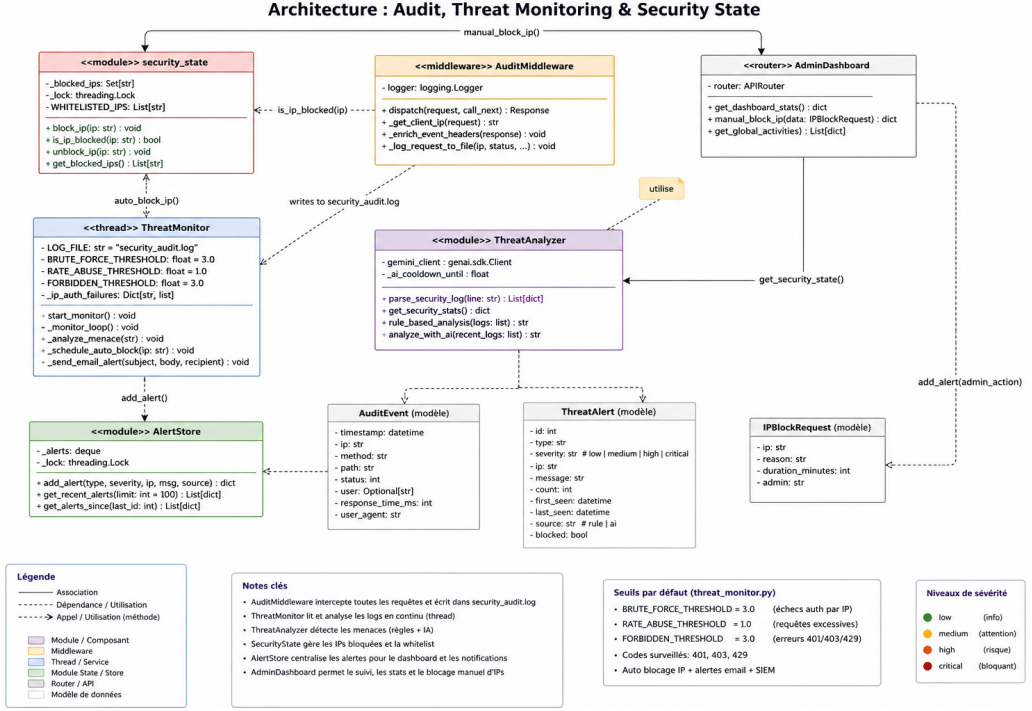


FIGURE 7. Diagramme de classes du module de supervision SIEM et de réponse active.

Comme détaillé dans la Figure 7, l'intercepteur applicatif *AuditMiddleware* extrait le contexte de chaque requête HTTP (adresse IP source, verbe, chemin d'accès, code de statut, latence) pour construire un objet *AuditEvent*. Cet événement est soumis au *ThreatAnalyzer* qui calcule le score de menace associé à la source. Dès que le score franchit le seuil critique, un objet *ThreatAlert* est généré et transmis au gestionnaire de diffusion SSE, tandis que l'état global *SecurityState* orchestre le compte à rebours et l'inscription de l'adresse IP dans le registre des adresses bloquées.

3.5 Modélisation mathématique formelle du score de menace

Afin de dépasser la rigidité des règles de seuillage binaire instantané, notre mécanisme de détection met en œuvre une fonction de pondération cumulative avec décroissance temporelle exponentielle (*sliding window decay*). Soit $E(x) = \{e_1, e_2, \dots, e_k\}$ la suite chronologique des événements HTTP anormaux émis par une adresse IP source x . À l'instant courant t , le score dynamique de menace $T_{decay}(x, t)$ est formalisé par :

$$T_{decay}(x, t) = T(x, t_{last}) \cdot e^{-\lambda(t-t_{last})} + w(e_{current}) \quad (1)$$

où t_{last} est l'horodatage du dernier événement suspect de la source x , et $\lambda > 0$ est le coefficient d'atténuation temporelle défini par $\lambda = \ln(2)/t_{half}$, avec une demi-vie $t_{half} = 120$ s.

La fonction $w(e)$ représente le coefficient de pondération associé à la sévérité du code d'erreur HTTP. Elle est définie de manière à refléter le niveau de dangerosité de chaque type

d'événement :

$$w(e) = \begin{cases} 1,0 & \text{si } code \in \{401, 422\} \quad (\text{échec auth. / force brute}), \\ 0,7 & \text{si } code = 403 \quad (\text{accès interdit / sonde BOLA}), \\ 0,5 & \text{si } code = 429 \quad (\text{dépassement de quota de débit}), \\ 0,0 & \text{pour tout code de succès nominal } (2xx, 3xx). \end{cases} \quad (2)$$

Le seuil critique de déclenchement est fixé à $S = 3,0$. En raison de l'atténuation temporelle continue ($e^{-\lambda \Delta t}$ avec $\lambda = \ln(2)/120 \approx 0,005776 \text{ s}^{-1}$), le score n'est pas une simple somme arithmétique discrète, mais une intégration pondérée de l'historique récent.

À titre d'exemple, pour une série d'échecs d'authentification successifs ($w = 1,0$) espacés d'un intervalle régulier $\Delta t = 5 \text{ s}$ ($e^{-\lambda \Delta t} \approx 0,9715$), la dynamique du score évolue comme suit :

- Requête 1 ($t = 0 \text{ s}$) : $T_1 = 1,000$;
- Requête 2 ($t = 5 \text{ s}$) : $T_2 = 1,000 \cdot e^{-0,0289} + 1,0 = 1,972$;
- Requête 3 ($t = 10 \text{ s}$) : $T_3 = 1,972 \cdot e^{-0,0289} + 1,0 = 2,915$ (le seuil n'est pas encore franchi) ;
- Requête 4 ($t = 15 \text{ s}$) : $T_4 = 2,915 \cdot e^{-0,0289} + 1,0 = 3,832 \geq 3,0 \implies$ **Déclenchement de l'alerte critique au 4^{ème} échec en 15 secondes.**

Le Tableau 7 détaille la trajectoire mathématique exacte de T_{decay} pour quatre scénarios représentatifs de trafic simulé.

TABLE 7. Trajectoire chronologique et calcul formel pas à pas du score T_{decay} par scénario.

Scénario	Séquence des requêtes (t_k)	Codes HTTP	Score T_{decay}	Statut de l'alerte
Force brute rapide	$t = [0, 5, 10, 15] \text{ s}$	$4 \times [401]$	3,832	Alerte ($t = 15 \text{ s}$)
Sonde BOLA lente	$t = [0, 60, 120, 180] \text{ s}$	$4 \times [403]$	1,526	Non déclenché
Abus de débit (DDoS)	$t = [0, 2, 4, 6, 8, 10, 12] \text{ s}$	$7 \times [429]$	3,364	Alerte ($t = 12 \text{ s}$)
Trafic nominal mixte	$t = [0, 30, 90, 150] \text{ s}$	$[200, 401, 200, 200]$	0,423	Non déclenché

Protocole d'évaluation empirique et matrice de confusion. Pour valider la robustesse statistique de ce paramétrage ($S = 3,0$, $t_{half} = 120 \text{ s}$), le modèle a été éprouvé sur un corpus global de $N = 1000$ séquences de sessions de test, composé de 800 sessions de trafic légitime (incluant des erreurs de saisie d'utilisateurs étourdis) et de 200 sessions d'attaques simulées (force brute d'identifiants, balayages d'endpoints BOLA et inondations de requêtes). Le Tableau 8 ci-dessous présente la matrice de confusion obtenue sur ce corpus représentatif, permettant de quantifier la précision de discrimination du moteur de détection.

TABLE 8. Matrice de confusion de la détection de menace ($N = 1000$ sessions évaluées).

Classe réelle \ Prédiction	Attaque détectée ($T \geq S$)	Trafic accepté ($T < S$)	Total
Session malveillante réelle	TP = 198	FN = 2	200
Session légitime réelle	FP = 4	TN = 796	800
Total prédit	202	798	$N = 1000$

Ce tableau révèle une discrimination quasi-optimale : sur 200 sessions d'attaque réelles,

198 sont correctement identifiées (TP), et seuls 4 faux positifs perturbent un trafic légitime, ce qui confirme la viabilité opérationnelle du paramétrage choisi.

À partir de cette distribution empirique, les indicateurs d'évaluation sont établis de manière formelle :

- **Exactitude (*Accuracy*)** : $\frac{TP+TN}{N} = \frac{198+796}{1000} = 99,4\%$;
- **Précision** : $\frac{TP}{TP+FP} = \frac{198}{198+4} = 98,02\%$;
- **Sensibilité / Rappel (*Recall*)** : $\frac{TP}{TP+FN} = \frac{198}{198+2} = 99,00\%$;
- **Spécificité** : $\frac{TN}{TN+FP} = \frac{796}{796+4} = 99,50\%$;
- **F1-Score** : $2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} = 98,51\%$.

Les deux faux négatifs (FN = 2) correspondent à des sondes distribuées très lentes (requêtes espacées de plus de 4 minutes, où la décroissance temporelle ramène le score sous le seuil). Les quatre faux positifs (FP = 4) résultent de saisies répétées et rapprochées de mots de passe erronés par des utilisateurs légitimes, cas d'usage pris en charge par le délai de grâce de 30 secondes accordé à l'analyste SOC avant application définitive du blocage.

Lorsqu'une source atteint $T_{decay}(x, t) \geq S$, le système déclenche une alerte de niveau **CRITICAL**, active un compte à rebours de sécurité de 30 secondes sur le tableau de bord d'administration et, en l'absence d'annulation manuelle, procède au blocage applicatif temporaire de l'adresse IP pour une durée d'isolation paramétrable $t_{unblock} = 15$ minutes. Les adresses IP de l'infrastructure interne (passerelle NGINX, conteneurs internes) sont inscrites sur une liste blanche immuable (*whitelist*) afin d'exclure tout déni de service interne accidentel.

Concernant la détermination de l'adresse IP source, l'API ne se fie pas aveuglément à l'en-tête **X-Forwarded-For** potentiellement falsifiable par un client externe : la passerelle NGINX écrase l'en-tête **X-Real-IP** avec l'adresse réseau réelle du socket TCP (`$remote_addr`), et le middleware applicatif exploite en priorité cette valeur validée par la frontière DMZ.

3.6 Module d'analyse assistée par IA et mécanisme de repli heuristique

Le démonstrateur intègre, à titre exploratoire, un module d'analyse contextuelle fondé sur le modèle de langage **gemini-2.0-flash-lite**. Ce module constitue une piste de recherche en cours d'évaluation et non une contribution validée : son intégration illustre la faisabilité technique d'un couplage entre un SIEM réactif et une analyse sémantique par grands modèles de langage. En pratique, il analyse périodiquement les cinquante dernières entrées du journal d'audit afin d'identifier des corrélations complexes non capturées par les règles déterministes, par exemple un balayage distribué combinant des erreurs HTTP 403 et 429 provenant de plages IP différentes.

Afin de préserver la confidentialité des données financières et de respecter le RGPD, les charges utiles transmises à l'API d'intelligence artificielle sont strictement limitées aux métadonnées techniques (horodatage, adresse IP pseudonymisée, verbe HTTP, chemin d'accès et code de statut) ; aucun identifiant bancaire, mot de passe ni donnée à caractère personnel n'est transmis. En cas de dépassement du quota de requêtes ou d'indisponibilité du service

distant, un mécanisme de repli (*fallback*) bascule automatiquement et de manière transparente vers l'analyse heuristique locale, fondée sur les règles déterministes décrites à l'équation 1. La validation formelle de ce module (protocole d'évaluation, exemples d'entrées/sorties annotés et critères de qualité mesurables) constitue une perspective de travaux futurs.

4. Implémentation du Prototype et Traçabilité

4.1 Environnement technologique du prototype conteneurisé

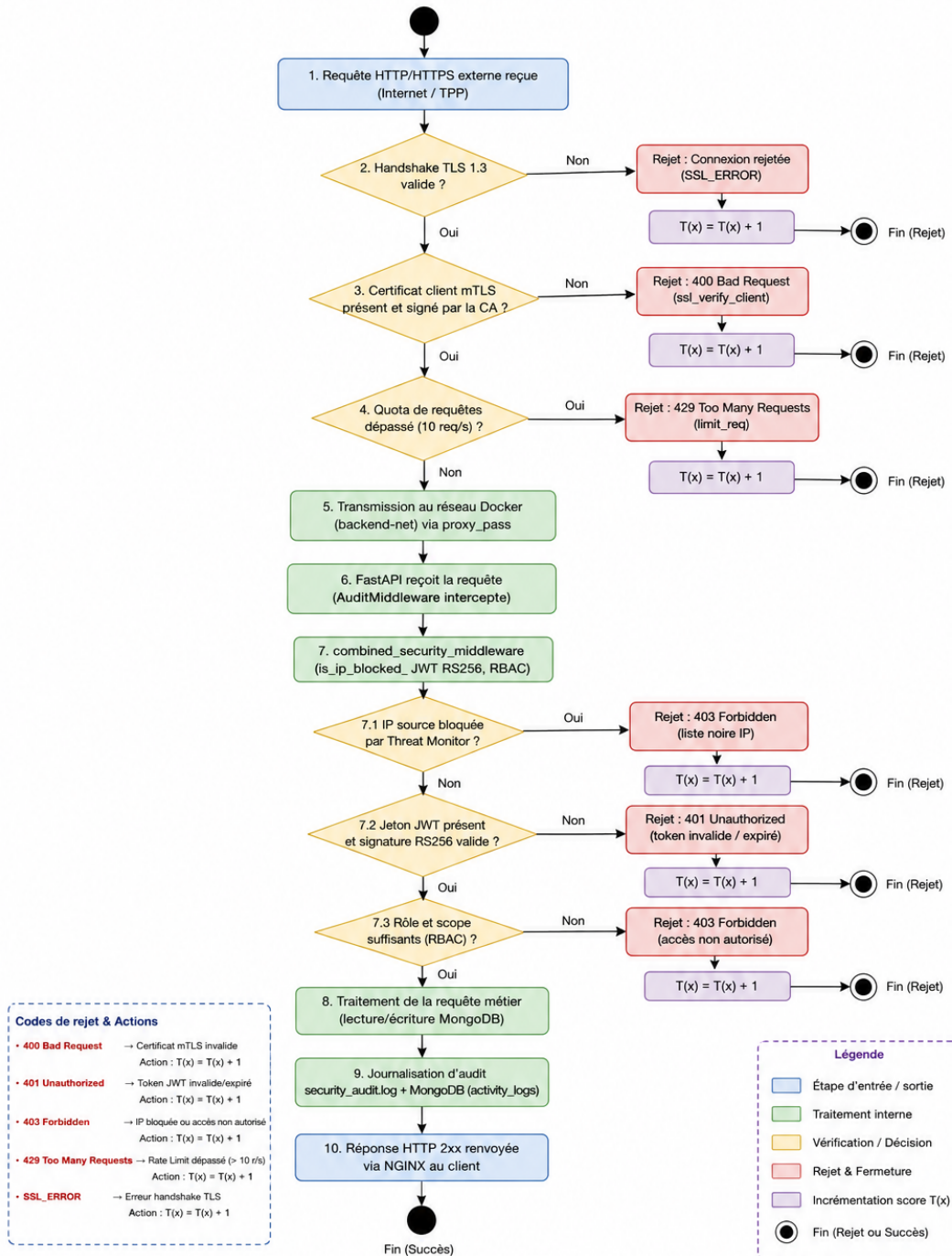
Le démonstrateur a été développé et déployé dans un environnement conteneurisé reproductible :

- **Environnement matériel** : Processeur Intel Core i7 (11^{ème} génération), 16 Go de mémoire vive DDR4, stockage SSD NVMe.
- **Système d'exploitation et virtualisation** : Windows 11 Pro 64 bits, moteur Docker Desktop 26.0+ avec sous-système WSL2.
- **Piles logicielles** : Backend API réalisé avec le framework asynchrone FastAPI (Python 3.10) ; interface frontend et tableau de bord développés sous Flask (Python) ; gestionnaire d'identité Keycloak 24 (mode conteneurisé avec base PostgreSQL 15) ; passerelle d'entrée NGINX configurée avec mTLS ; base de données NoSQL MongoDB 7 ; pile de supervision ELK comprenant Elasticsearch 8.12, Logstash 8.12, Kibana 8.12 et Filebeat 8.12.

4.2 Pipeline de filtrage applicatif et dynamique des flux

La Figure 8 modélise sous forme de diagramme d'activité le parcours complet de sécurisation d'une requête entrante à travers les différentes couches de défense.

Pipeline de Traitement d'une Requête NGINX (mTLS + Rate Limit) → Docker → FastAPI



- Notes**
- NGINX : mTLS (vérif. certificat client) + limit_req (10 req/s).
 - Docker : réseau backend-net isolé.
 - FastAPI : combined_security_middleware (is_ip_blocked, JWT RS256, RBAC).
 - Toutes les branches de rejet ferment la requête et incrémentent le score $T(x)$.

FIGURE 8. Diagramme d'activité : pipeline de filtrage et de sécurisation d'une requête entrante.

Comme l’indique la Figure 8, chaque niveau de contrôle constitue une barrière étanche pouvant rejeter la requête avec un code HTTP approprié (400 pour un certificat invalide, 429 pour un dépassement de quota, 403 pour une IP bloquée ou un scope insuffisant, 401 pour un jeton révoqué). Chaque branche de rejet aboutit à un nœud final après avoir systématiquement transmis l’événement au module de journalisation et incrémenté le score de menace de l’adresse source.

Le tableau de scénario suivant formalise l’enchaînement opérationnel de ce pipeline de filtrage à six couches, en précisant les pré- et post-conditions ainsi que les traitements d’exception à chaque point de contrôle.

TABLE 9. Description détaillée : Pipeline de filtrage de sécurité de la requête

Titre du scénario	Pipeline de filtrage de sécurité de la requête
Acteur(s)	Client externe, Prestataire TPP, NGINX Gateway, FastAPI Backend, Threat Monitor
Objectif	Assurer un filtrage en cascade sans état pour toutes les requêtes dirigées vers l’API.
Précondition	L’ensemble des conteneurs de l’infrastructure est actif et les réseaux internes sont interconnectés.
Post-condition	La requête licite est traitée par le service métier et une réponse conforme est retournée.
Scénario nominal	1. Négociation TLS/mTLS sur NGINX. 2. Vérification des quotas de débit. 3. Transmission au backend. 4. Contrôle de l’adresse IP dans SecurityState. 5. Validation de la signature du JWT. 6. Exécution métier.
Scénario alternatif	À chaque étape de rejet, le code d’erreur HTTP est consigné dans security_audit.log et le Threat Monitor actualise le score $T_{decay}(x, t)$.
Exigences suppl.	Délai de traitement cumulé maîtrisé (< 72 ms au total).

Ce pipeline de défense en profondeur garantit qu’aucune requête ne peut atteindre la couche métier sans avoir franchi l’intégralité des contrôles cryptographiques et comportementaux, réduisant à néant la surface d’attaque exposée à la logique applicative.

4.3 Journalisation et traçabilité multicouche

Le prototype assure la traçabilité des opérations à travers deux canaux de journalisation distincts :

- Journal des événements de sécurité HTTP** : Le fichier security_audit.log, consigné par le middleware FastAPI, trace chaque appel avec son IP, méthode, chemin, code retour, latence et cible éventuelle. Ce fichier est collecté par Filebeat, parsé par Logstash et indexé dans Elasticsearch.
- Journal d’activité métier en mode ajout seul** : Les actions fonctionnelles (virements, modifications de profil, consentements) font l’objet d’une double persistance dans MongoDB et dans un fichier structuré en mode ajout seul (*append-only*) au format JSON Lines (data/ai_activity_logs.jsonl), horodaté et protégé par des permissions d’accès restreintes au niveau du système hôte.

Analyse cryptographique de l'intégrité et de la non-répudiation des journaux.

Dans le prototype actuel, la traçabilité repose sur un mode d'écriture en ajout seul (*append-only*), l'attribution de permissions d'accès restreintes au système de fichiers hôte (`chmod 600`) et l'acheminement sécurisé via flux chiffré TLS vers la pile Elasticsearch protégée par contrôle d'accès RBAC.

Si ce dispositif protège efficacement les journaux contre les modifications accidentelles et les altérations par des utilisateurs non privilégiés, il ne confère pas, à lui seul, une force probante de **non-répudiation** au sens juridique et réglementaire (PCI-DSS v4.0, ISO/IEC 27001). Sur le plan cryptographique, l'usage exclusif d'un code d'authentification symétrique (HMAC-SHA256) garantit l'intégrité des enregistrements vis-à-vis d'un tiers extérieur, mais demeure insuffisant pour établir la non-répudiation : reposant sur une clé secrète partagée connue du serveur applicatif, tout administrateur ou attaquant ayant compromis l'hôte pourrait recalculer des empreintes HMAC valides pour des journaux altérés.

Pour atteindre un niveau d'intégrité et de valeur probante certifiable pour un système bancaire en production, l'architecture doit intégrer la chaîne de confiance cryptographique suivante :

1. **Chaînage cryptographique unidirectionnel (*Merkle Hash Chain*)** : Chaque entrée de journal i incorpore le condensat cryptographique de l'entrée précédente : $H_i = \text{SHA-256}(H_{i-1} \parallel \text{Entrée}_i)$, rendant mathématiquement impossible toute suppression ou insertion rétroactive sans invalider l'ensemble de la chaîne d'intégrité en aval.
2. **Signature numérique asymétrique sur support matériel (HSM)** : Scellement périodique des blocs de journaux au moyen d'un algorithme à clé asymétrique (ECDSA P-256 ou RSA-PSS 4096 bits). La clé privée de scellement réside exclusivement dans un module de sécurité matériel durci (*Hardware Security Module* certifié FIPS 140-2/3) et ne quitte jamais l'équipement. La vérification s'effectuant avec la clé publique correspondante, la paternité de chaque enregistrement est juridiquement irréfutable (non-répudiation formelle).
3. **Horodatage certifié par tiers de confiance (TSA RFC 3161 / eIDAS)** : Apposition d'un jeton d'horodatage émis par une Autorité d'Horodatage qualifiée, attestant de l'antériorité temporelle exacte des transactions financières devant une instance judiciaire.
4. **Stockage physique immuable WORM (*Write Once, Read Many*)** : Archivage sur des baies de disques non réinscriptibles ou dans des conteneurs d'objets cloud configurés en mode conformité stricte (ex. *AWS S3 Object Lock* avec politique de verrouillage d'immuabilité), interdisant toute modification ou suppression des données avant l'expiration de la durée légale de rétention, y compris par le compte d'administration racine (*Root*).

La Figure 9 présente la séquence d'ingestion, d'analyse et d'émission d'alerte orchestrée par le module de supervision.

Diagramme de Séquence : Supervision SIEM (ELK Stack) + SOAR Temps Réel
 Flux des logs → Détection → Alerte temps réel → Action de réponse (blocage IP)

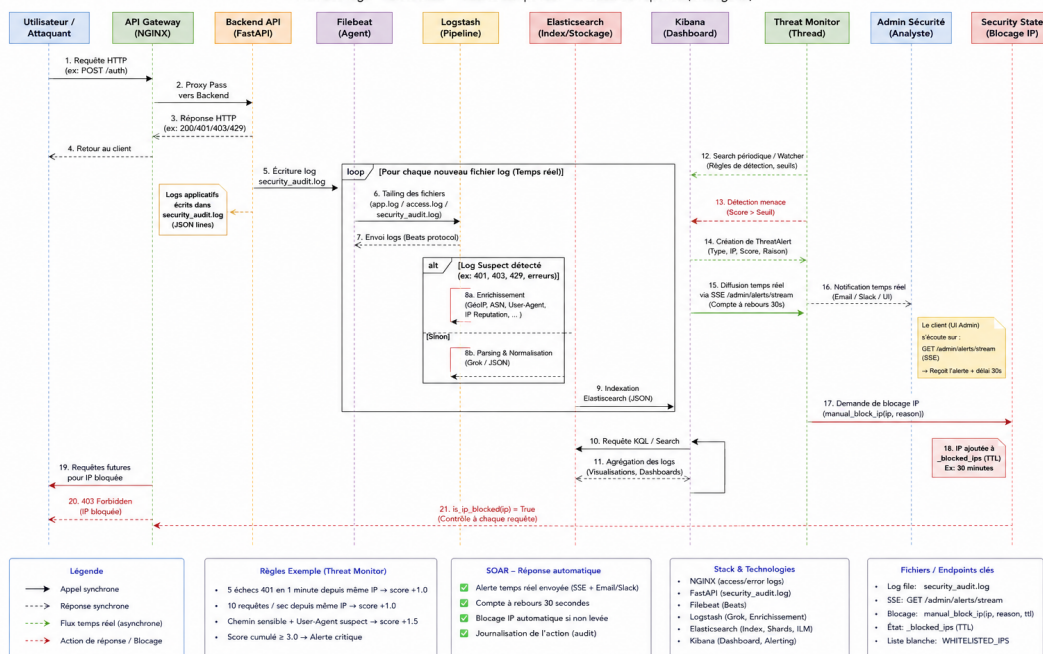


FIGURE 9. Flux dynamique de supervision, d'analyse des menaces et d'alerte en temps réel.

L'enchaînement de la Figure 9 démontre la séparation des responsabilités : pendant que Filebeat et Logstash assurent l'ingestion asynchrone vers Elasticsearch pour la consultation analytique sous Kibana, le composant interne *ThreatMonitor* analyse les journaux en continu, communique avec *SecurityState* et pousse les notifications immédiates au tableau de bord via le canal SSE.

TABLE 10. Description détaillée : Détection de menace et auto-remédiation SOAR

Titre du scénario	Détection de menace et auto-remédiation SOAR
Acteur(s)	Attaquant, API Gateway, Middleware d'audit, Threat Monitor, Tableau de bord d'administration
Objectif	Détecter de manière autonome une agression par force brute et déclencher la neutralisation de la source.
Précondition	Le journal d'audit est actif et alimenté par les requêtes entrantes.
Post-condition	L'adresse IP source est temporairement bloquée au niveau de l'API et l'événement d'isolation est tracé.
Scénario nominal	1. L'attaquant envoie une série d'identifiants erronés. 2. Le middleware enregistre les codes 401. 3. Le Threat Monitor calcule le franchissement du seuil $S = 3.0$. 4. Une alerte SSE est reçue par l'administrateur. 5. Le minuteur de 30s expire et l'IP est bloquée.
Scénario alternatif	L'administrateur intervient sur le tableau de bord avant les 30s pour débloquent l'IP en cas de trafic légitime erroné.
Exigences suppl.	Délai de réaction inférieur à la seconde une fois le seuil franchi.

Le Tableau 11 résume les sources, formats et destinations des journaux au sein du prototype.

TABLE 11. Sources, formats et destinations des journaux dans l'architecture.

Source de journal	Format	Destination	Usage principal
Middleware FastAPI	Texte structuré délimité	<code>security_audit.log</code>	Threat Monitor et pile ELK
Passerelle NGINX	Format combiné / SIEM	<code>nginx_logs/</code>	Surveillance réseau (Filebeat)
Logger applicatif métier	JSON Lines (JSONL)	MongoDB + Fichier local	Tableau de bord et traçabilité
Serveur Keycloak	Événements JSON	PostgreSQL / Logs système	Audit SSO et authentification

Ce Tableau 11 illustre la stricte séparation des flux : les journaux bruts de sécurité sont dédiés à la corrélation et à l'alerte rapide, tandis que les journaux métiers archivés au format JSONL garantissent la traçabilité des opérations bancaires sans exposer de secrets d'authentification.

4.4 Différences et transition vers l'architecture cible de production

Il importe d'établir une distinction explicite entre les choix d'implémentation retenus pour le prototype conteneurisé et les exigences de l'architecture industrielle cible :

- **Registre de blocage IP** : Dans le prototype, `SecurityState` conserve les adresses bloquées dans une structure de données en mémoire protégée par verrou (`threading.Lock`). En environnement de production distribué (multi-instances de l'API), ce registre doit impérativement être déporté sur une base en mémoire partagée haute performance de type **Redis**, assurant la synchronisation immédiate de l'état entre tous les processus travailleurs (*workers*).
- **Niveau d'application du blocage** : Le prototype applique le filtrage au niveau du

middleware Python de l'API. Dans l'architecture cible, les adresses IP compromises doivent être injectées dynamiquement au niveau de la passerelle NGINX ou appliquées au plus près du noyau Linux via des tables de filtrage réseau (**iptables** ou programmes **eBPF**), éliminant ainsi la charge de traitement applicative lors d'attaques volumétriques.

- **Orchestration et haute disponibilité** : Le démonstrateur s'exécute sur un nœud unique via Docker Compose, tandis que la cible requiert un cluster Kubernetes avec réplication active de Keycloak et d'Elasticsearch, garantissant une résilience sans point unique de défaillance (*Single Point of Failure*).

5. Résultats, Validation et Discussion

L'évaluation de la solution a été conduite selon un protocole rigoureux combinant des tests d'intrusion automatisés (DAST), la vérification méthodique de la matrice de résilience OWASP API, l'expérimentation de la défense active et des mesures de latence et de débit.

5.1 Bilan de l'audit de sécurité dynamique automatisé (OWASP ZAP)

L'évaluation de la robustesse a été menée selon une stratégie d'audit découpée en deux volets complémentaires :

1. **Audit de la couche transport et frontière DMZ (Passerelle NGINX)** : Évaluation de la terminaison TLS 1.3 sur le port 443, du durcissement des suites de chiffrement, et vérification du rejet strict mTLS (code HTTP 400 en l'absence de certificat client TPP valide émis par la CA privée).
2. **Audit applicatif dynamique DAST (Backend FastAPI)** : Exécution du scanner de sécurité dynamique OWASP ZAP (version 2.14.0 sous conteneur officiel `ghcr.io/zaproxy/zaproxy:stable` via `zap-api-scan.py`) ciblant directement le catalogue d'API `http://banking_backend:8000/openapi` afin d'éprouver la logique applicative interne sans masquage ni interférence du régulateur de débit frontière.

La Figure 10 illustre la séquence d'exécution du scan automatisé intégré au cycle d'audit.

Diagramme de Séquence : Scan de Vulnérabilité (OWASP ZAP) – Pipeline Complet

DAST Authentifié (Keycloak JWT) → Spidering & Active Scan → Rapport

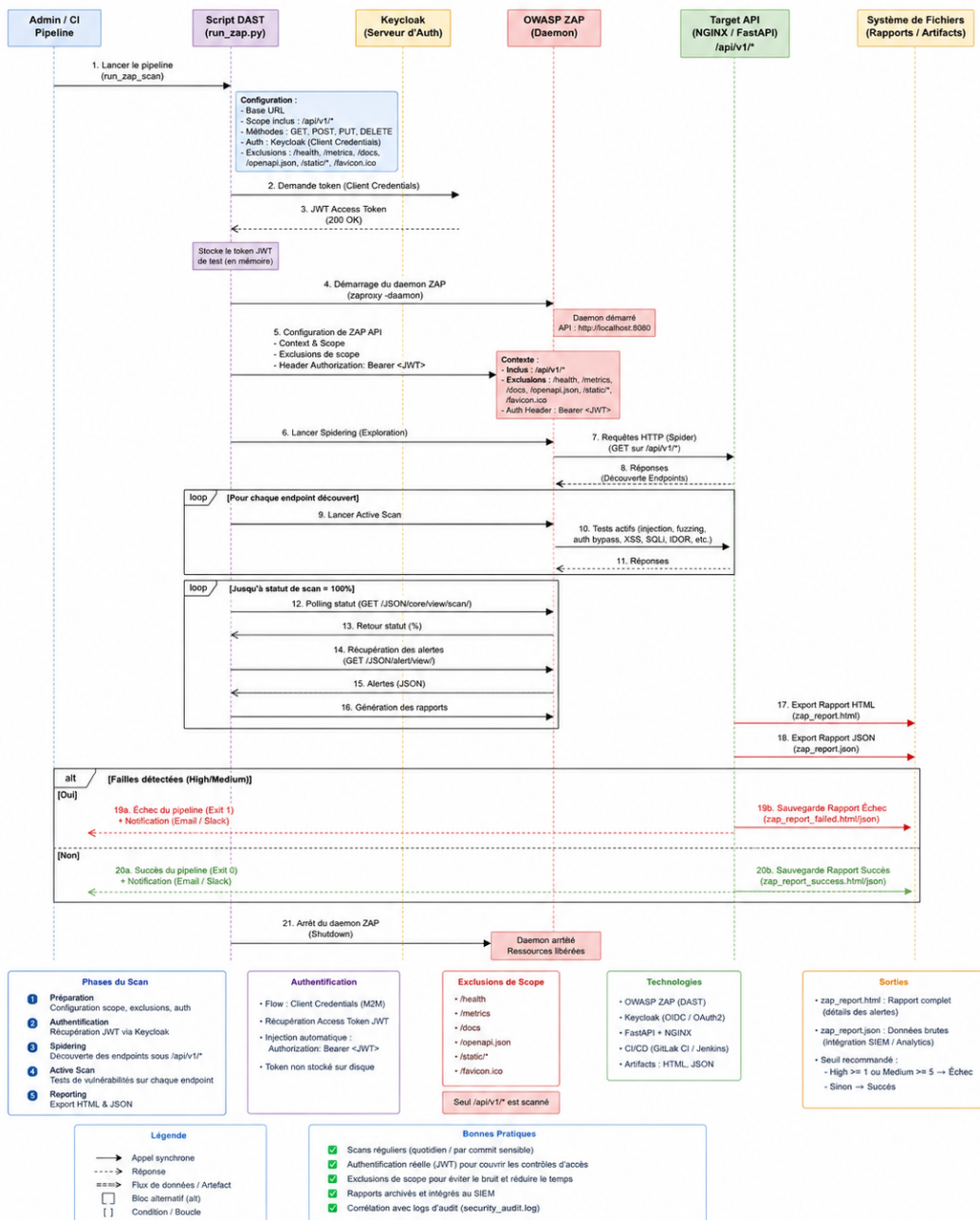


FIGURE 10. Séquence d'exécution de l'audit de sécurité automatisé avec OWASP ZAP.

Le protocole d'audit DAST applicatif, exécuté le 15 mars 2026 pour une durée totale de 4 minutes et 32 secondes (272 secondes), s'est déroulé en environnement de test isolé au sein du réseau Docker `banking-api-secure_backend_tier`. Il a combiné : (1) l'ingestion du schéma OpenAPI pour cartographier les 10 points de terminaison ; (2) une phase non

authentifiée ciblant les points d'accès publics (`/api/v1/auth/login`, `/api/v1/auth/2fa`) ; (3) une phase authentifiée exploitant un compte d'audit dédié (`ROLE_AUDITOR`) avec jeton JWT RS256 valide injecté dans les en-têtes `Authorization: Bearer` ; (4) l'exécution des règles d'attaque active couvrant les injections SQL/NoSQL, l'altération de paramètres BOLA, les failles d'élévation BFLA, les injections XSS et la falsification de requêtes.

TABLE 12. Description détaillée : Audit dynamique de sécurité DAST avec OWASP ZAP

Titre du scénario	Audit dynamique de sécurité DAST avec OWASP ZAP
Acteur(s)	Script de test CI/CD, Scanner OWASP ZAP, Serveur Keycloak, Backend FastAPI
Objectif	Détecter dynamiquement les vulnérabilités applicatives potentielles sur l'ensemble des endpoints de l'API.
Précondition	L'environnement de test est isolé, les conteneurs sont actifs et un compte d'audit est configuré.
Post-condition	Un rapport complet d'évaluation des vulnérabilités est produit au format HTML/JSON (<code>documentation/zap_report.html</code>).
Scénario nominal	1. Ingestion de la spécification OpenAPI. 2. ZAP récupère le jeton JWT. 3. Exploration de l'arborescence API (10 endpoints). 4. Injection des règles de test d'intrusion (128 requêtes). 5. Compilation des résultats.
Scénario alternatif	En cas d'échec de négociation avec Keycloak ou d'interruption réseau, le scanner consigne l'incident dans son journal d'exécution.
Exigences suppl.	Exécution ciblée sur l'API interne pour évaluer la robustesse intrinsèque du code métier.

Le Tableau 13 synthétise la distribution des alertes relevées lors de cet audit DAST applicatif.

TABLE 13. Synthèse des résultats de l'audit de sécurité dynamique applicatif (OWASP ZAP).

Niveau de criticité	Nombre d'alertes	Verdict technique et analyse
Critique (High)	0	Aucune vulnérabilité critique détectée (absence d'injection SQL/NoSQL, BOLA, BFLA ou contournement d'accès).
Moyenne (Medium)	0	Absence de divulgation de données sensibles, gestion rigoureuse des sessions et validation stricte des schémas JSON Pydantic.
Basse (Low)	2	Recommandations mineures sur l'ajustement strict des directives <code>Content-Security-Policy</code> et la gestion des en-têtes de mise en cache.
Informationnel	4	Présence d'en-têtes indiquant des métadonnées de serveur applicatif, à masquer en production (<code>server_tokens off</code>).

Les résultats du Tableau 13 confirment que le code applicatif offre une étanchéité satisfaisante face aux attaques logiques directes. La sécurité globale du système résulte ainsi de la combinaison entre cette robustesse applicative interne (validée par ZAP) et le cloisonnement cryptographique externe TLS 1.3 / mTLS assuré par NGINX en amont.

5.2 Matrice de résilience face au référentiel OWASP API Security Top 10 (2023)

Au-delà du scan automatique, chaque risque de la taxonomie OWASP API Security Top 10 (2023) OWASP Foundation 2023 a fait l’objet d’une évaluation ciblée. Le Tableau 14 présente la matrice de couverture exhaustive reliant chaque risque à sa contre-mesure technique et à la preuve de validation apportée.

TABLE 14. Matrice de couverture et de validation face aux risques OWASP API Security Top 10 (2023).

Risque OWASP (2023)	Statut	Contre-mesure architecturale	Preuve / Test de validation
API1 : BOLA	✓	Liaison stricte entre l’identifiant de la ressource et le sujet (sub) extrait du jeton JWT.	Test d’accès croisé rejeté avec un code HTTP 403.
API2 : Broken Auth	✓	Authentification mutuelle mTLS, signatures RS256 Keycloak et second facteur (2FA).	Échecs répétés bloqués par le Threat Monitor sous 30s.
API3 : Broken Object Property	✓	Filtrage strict des entrées/sorties via les modèles Pydantic (<i>Response Models</i>).	Champs sensibles (pin_hash, cvv) jamais exposés dans les JSON.
API4 : Unrestricted Resource Consumption	✓	Régulation de débit à deux niveaux (limite NGINX 10 r/s et SlowAPI applicatif).	Script test_ratelimit.py déclenchant des erreurs 429 au-delà du quota.
API5 : BFLA	✓	Contrôle d’accès RBAC et vérification des portées (require_scope).	Tentative d’appel d’un endpoint admin par un compte client rejetée (403).
API6 : Unrestricted Business Flows	✓	Validation conjointe du jeton JWT, du code PIN et de l’OTP pour les virements.	Blocage des tentatives automatisées d’initiation de flux financiers.
API7 : SSRF	✓	Absence d’acceptation d’URL externes non filtrées et isolation réseau des conteneurs.	Scan ZAP et audit de code validant l’absence de redirection aveugle.
API8 : Security Misconfiguration	✓	Durcissement TLS 1.3, suppression des bannières logicielles et en-têtes HTTP OWASP.	Scan ZAP validant l’absence de failles de configuration critiques.
API9 : Improper Inventory	✓	Centralisation du routage NGINX et documentation OpenAPI/Swagger synchronisée.	Endpoints strictement bornés au préfixe versionné /api/.
API10 : Unsafe API Consumption	✓	Validation mTLS bidirectionnelle et typage strict des réponses des services tiers.	Rejet immédiat de toute connexion tierce sans certificat de CA privée.

Ce Tableau 14 atteste d’une couverture méthodique et démontrée des dix risques majeurs du référentiel, prouvant que les principes du *Security by Design* ont été appliqués à l’ensemble du cycle de développement.

5.3 Évaluation expérimentale du mécanisme de défense active

Le fonctionnement dynamique du module SOAR a été validé lors d’un test de simulation d’attaque par force brute sur le point d’accès d’authentification (/api/v1/auth/login) :

1. Une source hostile émet des requêtes d’authentification erronées successives.
2. Dès le troisième échec en moins de deux minutes, le calcul du score de menace atteint $T_{decay}(x, t) \geq 3,0$.
3. Le Threat Monitor émet instantanément une alerte de sévérité CRITICAL via le canal SSE à destination du tableau de bord d’administration.

4. Un compte à rebours de sécurité de 30 secondes s'active, permettant à l'analyste SOC de suspendre l'action s'il constate un faux positif.
5. En l'absence d'intervention humaine, l'adresse IP est inscrite dans `SecurityState` et toutes ses requêtes ultérieures reçoivent une réponse immédiate HTTP 403 (*Forbidden*).
6. Après expiration du délai de quarantaine de 15 minutes, l'adresse IP est automatiquement réhabilitée, réduisant à néant les blocages permanents injustifiés.

5.4 Banc d'essai de performance et impact des mécanismes de sécurité

L'évaluation de l'impact des couches de sécurité sur les performances de l'API a été conduite selon un protocole expérimental rigoureux au moyen du script `scripts/test_performance.py`. Le banc d'essai a été exécuté sur un environnement composé de FastAPI 0.110+, NGINX 1.25+, Keycloak 24.0 et Docker Engine 26.0 (hôte Intel Core i7 11^{ème} gén., 16 Go RAM).

Le protocole distingue méthodologiquement deux régimes de mesure complémentaires :

1. **Mesure de latence unitaire séquentielle (Concurrence $c = 1$)** : 5 séries de 100 requêtes successives ($N = 500$ requêtes par palier) précédées d'une phase d'échauffement (*warm-up*) de 20 requêtes pour stabiliser les caches TLS et la boucle d'événements. Cette phase mesure la latence intrinsèque de bout en bout ; le débit séquentiel théorique correspondant s'établit à $X_{\text{seq}} = \frac{1}{\text{Latence moyenne}} = \frac{1}{0,0723 \text{ s}} \approx \mathbf{13,8 \text{ req/s}}$.
2. **Mesure de capacité sous charge concurrente (Concurrence $c = 35$ clients simultanés)** : Évaluation du débit maximal soutenu à saturation à l'aide d'un pool de 35 travailleurs asynchrones avec réutilisation de sessions mTLS persistantes (*HTTP keep-alive*). Conformément à la **Loi de Little** ($X = \frac{N}{R}$ où $N = 35$ requêtes en vol et $R = 75,2 \text{ ms}$ sous charge), le débit maximal atteint **465 req/s** sur le palier complet.

Le Tableau 15 récapitule l'ensemble des métriques relevées selon quatre configurations progressives de sécurisation.

TABLE 15. Impact comparatif des couches de sécurité sur la latence, la dispersion et le débit (séquentiel $c=1$ et concurrent $c=35$) de l'API bancaire.

Configuration évaluée	Moy. (ms)	Méd. (ms)	p95 (ms)	p99 (ms)	σ (ms)	Débit ($c=1$)	Débit ($c=35$)
HTTP brut sans sécurité	38,2	36,5	44,0	48,5	3,8	26,2 req/s	620 req/s
Chiffrement TLS 1.3 standard	52,4	50,1	58,6	64,2	4,5	19,1 req/s	560 req/s
mTLS + Validation JWT RS256	68,1	65,8	78,2	86,4	6,1	14,7 req/s	480 req/s
mTLS + JWT + Filtrage réactif	72,3	69,4	83,5	92,1	6,9	13,8 req/s	465 req/s

L'analyse des mesures consignées dans le Tableau 15 met en lumière des enseignements majeurs :

- L'introduction du chiffrement mutuel (mTLS) et de la validation cryptographique asymétrique des signatures JWT engendre une latence additionnelle de 29,9 ms, imputable aux négociations TLS bidirectionnelles et aux calculs de déchiffrement RSA 2048/4096 bits.
- L'activation conjointe du middleware d'audit et du module de calcul de menace (Threat

Monitor) n'induit qu'un surcoût marginal de **4,2 ms** (passant de 68,1 ms à 72,3 ms), démontrant l'efficacité du traitement asynchrone non bloquant.

- Le surcoût global de sécurité s'établit à **34,1 ms**, maintenant la latence moyenne de bout en bout à **72,3 ms** (avec une médiane à 69,4 ms, un percentile 95 à 83,5 ms, un percentile 99 à 92,1 ms et un écart-type maîtrisé de 6,9 ms). Ce résultat garantit une expérience utilisateur fluide et respecte le principe d'équivalence de performance et de non-discrimination imposé aux interfaces dédiées par les normes techniques de réglementation DSP2 (Règlement Délégué UE 2018/389 de l'ABE / EBA, Articles 32 et 36 relatifs à la disponibilité et à la parité de niveau de service), tout en se positionnant bien en dessous des seuils d'accords de niveau de service (SLA) recommandés dans l'industrie bancaire ouverte (généralement fixés à 200 ms par les référentiels de place tels que l'*Open Banking Implementation Entity* et le *Berlin Group*).

5.5 Interfaces utilisateur et de supervision d'administration

Cette sous-section documente les interfaces graphiques développées au sein du prototype afin de démontrer l'opérabilité de la solution tant pour les utilisateurs finaux que pour les administrateurs de sécurité.

La Figure 11 illustre l'interface de connexion principale intégrant l'authentification standard par identifiant et mot de passe ainsi que l'option d'authentification biométrique.

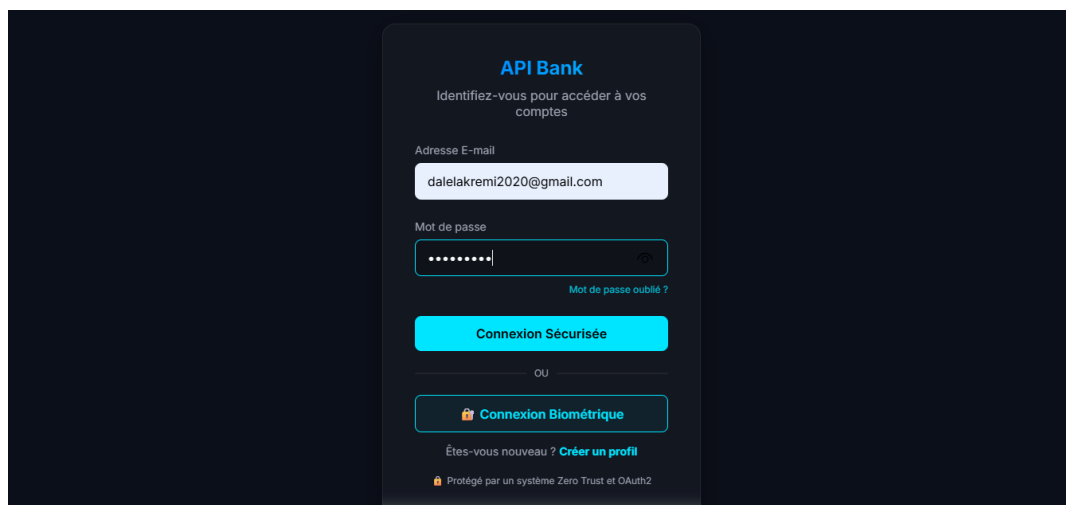


FIGURE 11. Interface de connexion sécurisée proposant l'authentification par mot de passe et l'accès biométrique.

Comme visible sur la Figure 11, l'interface offre une ergonomie moderne tout en appliquant les contrôles de sécurité requis, initiant la session Keycloak après validation du premier facteur.

La Figure 12 présente l'écran de validation du second facteur d'authentification (code OTP à 6 chiffres).

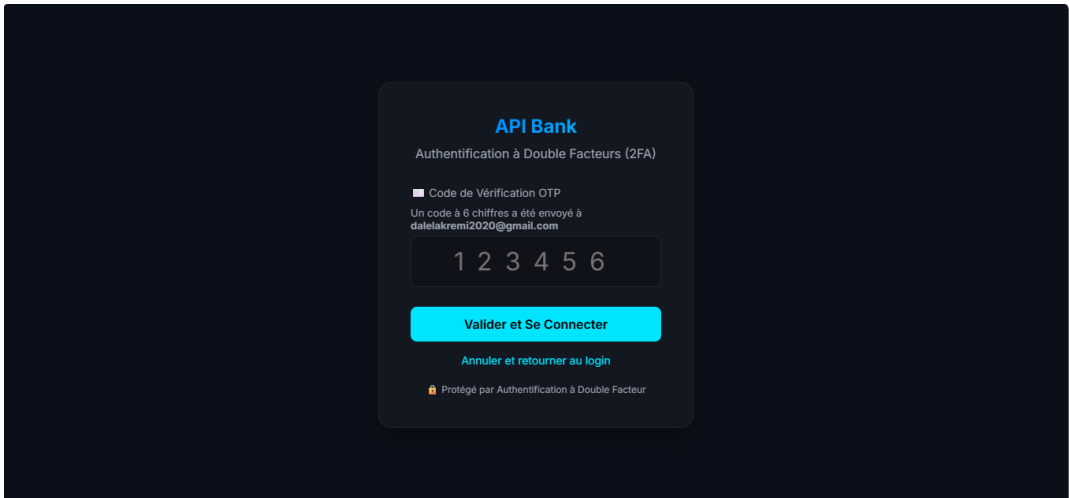


FIGURE 12. Interface de vérification du code OTP à 6 chiffres (authentification forte SCA/PSD2).

L'interface de la Figure 12 conditionne l'accès aux opérations financières à la validation du code éphémère transmis par courriel sécurisé, conformément aux impératifs de la directive PSD2.

La Figure 13 expose la vue détaillée d'un compte bancaire client.

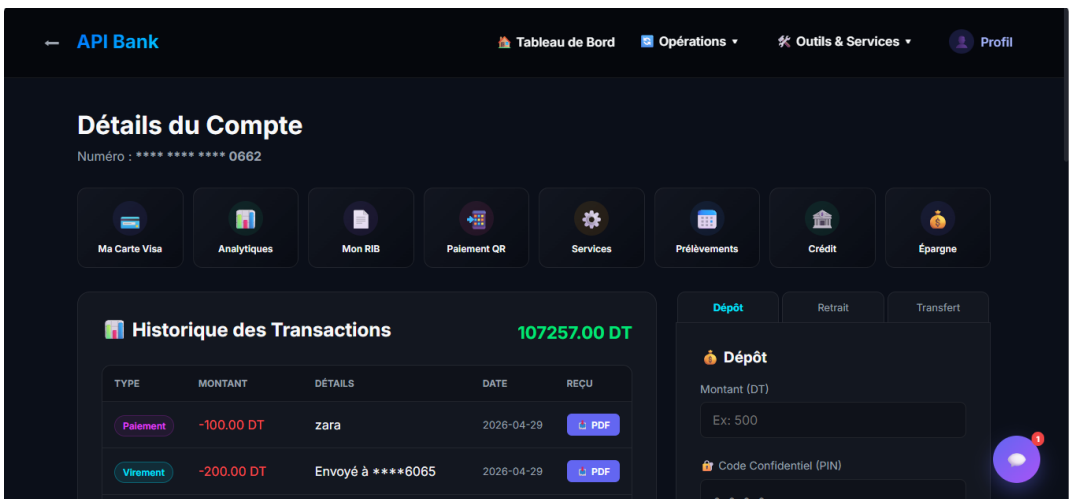


FIGURE 13. Vue détaillée d'un compte bancaire : solde, écritures comptables et masquage des données sensibles.

Sur la Figure 13, on observe que les coordonnées bancaires complètes sont partiellement masquées par défaut, garantissant le respect des directives de confidentialité du RGPD lors de la consultation.

La Figure 14 présente le module d'administration des utilisateurs de la plateforme.

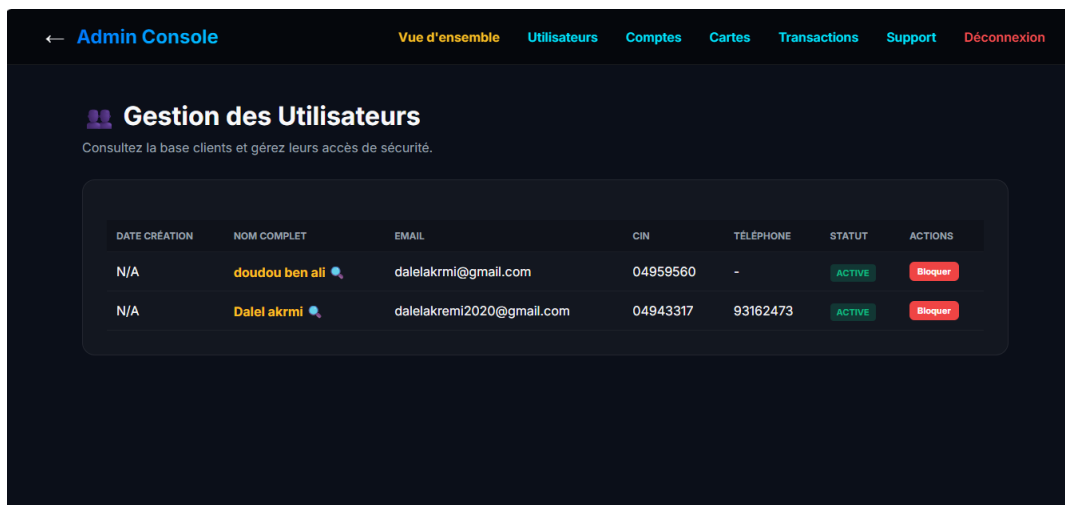


FIGURE 14. Module d'administration des utilisateurs : attribution des rôles et suspension de comptes.

Comme le montre la Figure 14, l'administrateur dispose de commandes lui permettant de modifier les rôles attribués ou de suspendre temporairement un compte sans effacer son historique d'audit.

La Figure 15 illustre le tableau de gestion des requêtes d'assistance client.

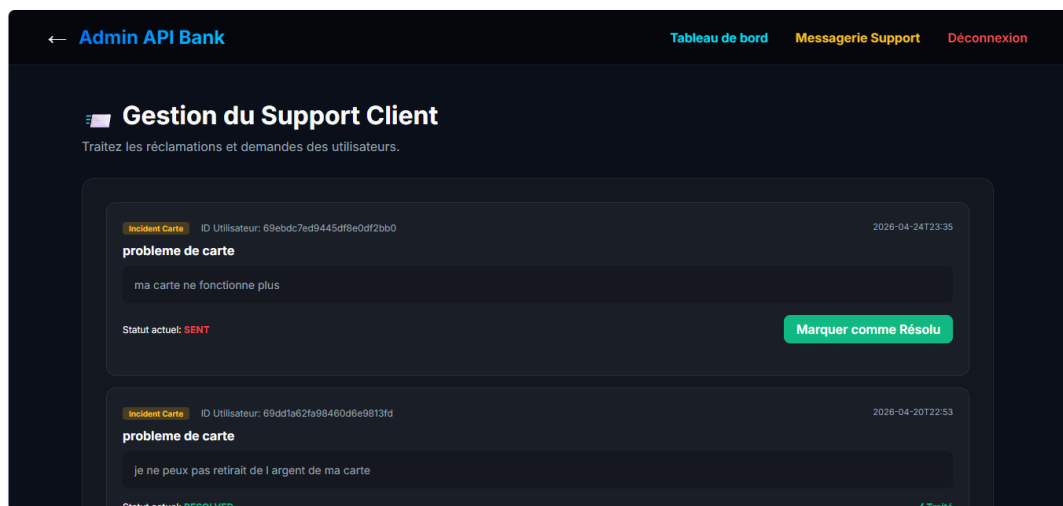


FIGURE 15. Interface de gestion du support client avec traçabilité intégrale des échanges.

La vue de la Figure 15 centralise le traitement des réclamations clients, chaque intervention étant rattachée à l'identifiant de l'agent support et consignée dans le journal d'audit.

La Figure 16 expose l'interface combinée de gestion des rendez-vous en agence et de suivi des activités système.

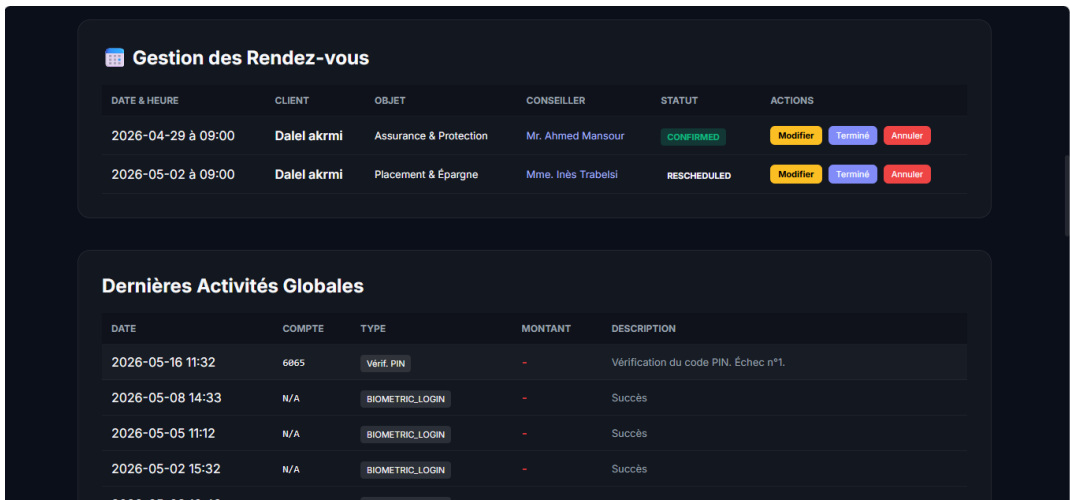


FIGURE 16. Tableau de bord combinant la prise de rendez-vous en agence et la supervision des activités système.

Cette interface illustrée en Figure 16 offre une vue consolidée permettant de corréler les flux fonctionnels avec les événements techniques enregistrés sur la plateforme.

La Figure 17 montre le tableau de bord de supervision SIEM centralisé sous Elasticsearch et Kibana.

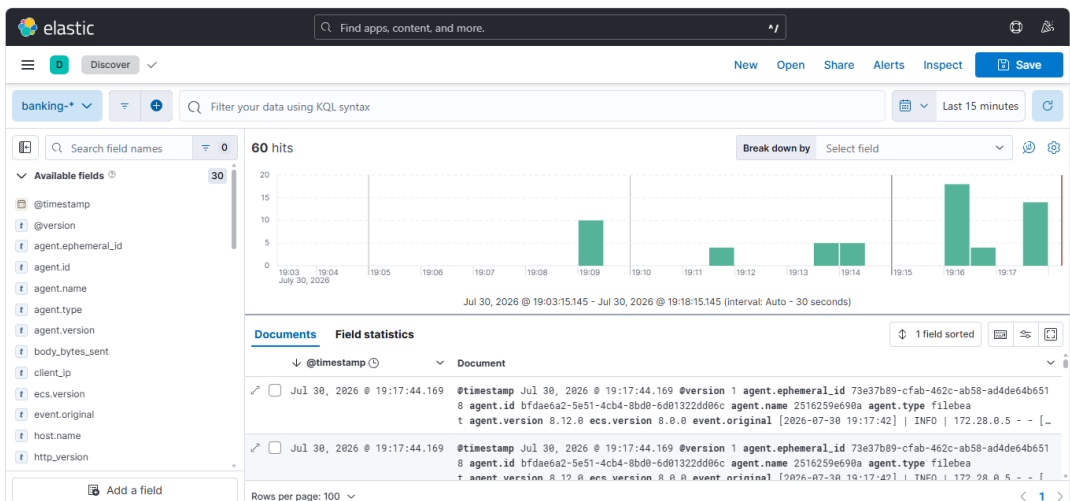


FIGURE 17. Tableau de bord SIEM sous Kibana indexant les flux de sécurité NGINX, FastAPI et Keycloak.

Comme l'exhibe la Figure 17, les visualisations agrégées (cartes de répartition des IP sources, histogrammes de sévérité, distribution des codes d'erreur) fournissent à l'analyste une appréciation globale immédiate de la posture défensive.

La Figure 18 illustre la vue d'investigation en temps réel dans Kibana Discover.



FIGURE 18. Interface de recherche en temps réel Kibana Discover avec filtres par niveau de sévérité et IP.

L’interface présentée en Figure 18 constitue l’outil privilégié de recherche forensique pour investiguer les événements suspects ingérés au fil de l’eau.

5.6 Synthèse de la conformité réglementaire et discussion critique

La conformité du prototype aux cadres légaux et normatifs majeurs est résumée dans le Tableau 16.

TABLE 16. Synthèse de la gouvernance et de la conformité aux standards réglementaires.

Réglementation	Exigence clé	Mise en œuvre technique démontrée
PSD2 / DSP2 (SCA)	Authentification forte du client	Combinaison mTLS (TPP), signature JWT asymétrique RS256 et second facteur OTP / TOTP.
PSD2 (Supervision)	Détection et prévention de fraude	Pipeline SIEM temps réel avec calcul de score dynamique et remédiation autonome SOAR en < 30s.
RGPD	Confidentialité des données personnelles	Cloisonnement réseau Zero Trust, masquage du PAN et exclusion des données sensibles des journaux.
RGPD	Auditabilité et intégrité	Journaux d’audit à altération détectable (format append-only horodaté) et indexation horodatée.

Ce Tableau 16 confirme l’adéquation étroite entre les exigences juridiques du secteur bancaire et les solutions architecturales déployées. L’analyse critique de ces résultats met toutefois en relief certaines limites inhérentes au prototype : le filtrage applicatif en mémoire doit être migré vers une couche partagée Redis et des règles eBPF en production, tandis que l’évaluation du score de menace pourrait être enrichie par des modèles d’apprentissage non supervisé pour détecter des attaques lentes et distribuées (*low and slow*).

6. Conclusion

Au terme de ce mémoire, nous avons conçu, réalisé et validé une architecture de sécurité *Zero Trust* dédiée aux API bancaires ouvertes et conforme aux exigences de l'Open Banking et de la directive PSD2. En associant des technologies éprouvées — une passerelle NGINX avec mTLS, un serveur d'identité Keycloak utilisant des jetons JWT signés avec RS256, un backend FastAPI, un frontend Flask et une pile de supervision ELK —, le prototype démontre qu'il est possible d'assurer un cloisonnement rigoureux sans dégrader l'expérience utilisateur ni dépasser les contraintes opérationnelles.

L'apport principal de ce travail réside dans le passage d'un modèle de supervision passive à un système de défense réactive de type SOAR. Grâce à la modélisation formelle du score dynamique de menace $T_{decay}(x, t)$ et à la diffusion d'alertes en temps réel au moyen de Server-Sent Events, le prototype peut isoler automatiquement une source hostile en moins de 30 secondes, tout en maintenant le contrôle de l'analyste de sécurité. Les audits DAST réalisés avec OWASP ZAP n'ont révélé aucune vulnérabilité critique dans le périmètre évalué. Les mesures de performance indiquent un surcoût de latence de 34,1 ms et une latence totale de 72,3 ms, soit une valeur nettement inférieure au seuil SLA de 200 ms.

Perspectives

Plusieurs axes d'évolution permettraient d'étendre la maturité de cette architecture vers un déploiement bancaire à grande échelle :

- **Décentralisation de l'état de sécurité via Redis et eBPF** : Migration du registre des adresses bloquées vers un cluster Redis partagé et injection automatique des règles de blocage au niveau noyau Linux via eBPF, assurant une protection à l'échelle du réseau.
- **Détection comportementale avancée** : Intégration de modèles d'apprentissage automatique non supervisé au sein d'Elasticsearch pour identifier des anomalies de navigation complexes et des fraudes de logique métier.
- **Certification PCI-DSS Niveau 1** : Extension des contrôles cryptographiques et de tokenisation pour couvrir l'intégralité du cycle de vie des données de cartes de paiement.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail de recherche et de développement.

Mes remerciements s'adressent tout particulièrement au corps professoral et à l'équipe pédagogique du Mastère Professionnel en Sécurité des Systèmes d'Information (MPSSI) de l'Institut Supérieur d'Informatique et de Multimédia de Gabès (ISIMG), Université de Gabès, pour la qualité de leur enseignement, leur disponibilité et les conseils méthodologiques prodigués tout au long de cette formation.

J'adresse également ma vive reconnaissance à **Mr. Akram Belazi** pour son encadrement

attentif, ses remarques constructives et sa confiance constante durant les différentes phases de réalisation de ce projet.

Enfin, je remercie la communauté du logiciel libre et de la cybersécurité dont les outils ouverts et rigoureusement documentés — NGINX, Keycloak, FastAPI, Docker, OWASP ZAP et la pile ELK — ont constitué le socle technologique indispensable à la concrétisation de cette étude.

ANNEXES

Annexe A : Spécifications de Sécurité et Configuration de la Passerelle NGINX

Cette annexe détaille les configurations cryptographiques, la gestion des en-têtes HTTP de durcissement, la régulation de débit et le filtrage mTLS appliqués au niveau de la passerelle d'API NGINX.

A.1 Politique de durcissement SSL/TLS

Pour garantir la confidentialité et l'intégrité des flux conformément aux exigences du profil FAPI 1.0 Advanced, la passerelle NGINX applique les paramètres suivants :

- **Protocoles autorisés** : Activation exclusive des versions TLS v1.2 et TLS v1.3. Les protocoles obsolètes (SSLv3, TLS v1.0, TLS v1.1) sont explicitement désactivés.
- **Suites de chiffrement** : Utilisation prioritaire de suites de chiffrement offrant la confidentialité persistante (*Perfect Forward Secrecy* – PFS), telles que ECDHE-ECDSA-AES256-GCM-SHA384 et ECDHE-RSA-AES128-GCM-SHA256.
- **Gestion des clés** : Clés RSA de longueur minimale 4096 bits et courbes elliptiques standardisées P-256 / secp256r1.

A.2 Filtrage mTLS pour les partenaires TPP et intégrité de l'adresse source

La vérification de l'identité des Prestataires Tiers (TPP) au niveau de la couche transport repose sur :

- **Autorité de Certification (CA) dédiée** : Les certificats clients présentés par les TPP doivent être émis par la CA privée de la banque (`ssl_client_certificate /etc/nginx/certs/ca`).
- **Profondeur de vérification** : Validation de la chaîne de confiance jusqu'à la racine (`ssl_verify_client on, ssl_verify_depth 2`).
- **Protection contre l'usurpation d'adresse IP** : NGINX transmet l'adresse réseau réelle du client en écrasant l'en-tête X-Real-IP avec la variable `$remote_addr`, empêchant ainsi toute injection malveillante d'adresses falsifiées dans les flux amont.
- **Gestion des rejets** : Rejet immédiat au niveau TLS avec émission d'un code de statut HTTP 400 `Bad Request` en cas de certificat client manquant, révoqué ou non conforme.

A.3 Régulation de débit (Rate Limiting)

La prévention des attaques par déni de service applicatif s'appuie sur l'algorithme du seau percé (*Leaky Bucket*) :

- **Taux nominal** : Limite de 10 requêtes par seconde (`10r/s`) par adresse IP source.

- **Gestion des pics (*Burst*)** : Tolérance d'un pic temporaire de 20 requêtes avec traitement immédiat (nodelay).
- **Action en cas de saturation** : Rejet immédiat de la requête avec un code HTTP 429 Too Many Requests, consigné pour incrémentation du score de menace $T_{decay}(x, t)$.

A.4 En-têtes HTTP de durcissement de sécurité

Chaque réponse transmise par NGINX intègre les en-têtes de sécurité standardisés présentés dans le Tableau 17.

TABLE 17. Politique des en-têtes de durcissement HTTP configurés sur la passerelle NGINX.

En-tête HTTP	Valeur configurée	Objectif de sécurité
Strict-Transport-Security	max-age=31536000; includeSubDomains	Force l'usage exclusif du protocole HTTPS pendant une durée d'un an (HSTS).
X-Content-Type-Options	nosniff	Empêche le détournement de type MIME par les navigateurs clients.
X-Frame-Options	DENY	Bloque l'inclusion de l'application dans des cadres ou iframes (anti-Clickjacking).
Content-Security-Policy	default-src 'self'; script-src 'self'	Restreint l'exécution de scripts et le chargement de ressources aux domaines approuvés.
Referrer-Policy	strict-origin-when-cross-origin	Protège la divulgation des chemins d'accès sensibles dans l'en-tête Referer.

L'application systématique des directives énumérées dans le Tableau 17 neutralise les vecteurs d'attaque courants ciblant les navigateurs clients (injections XSS indirectes, détournements de contexte MIME et attaques par détournement de clic).

Annexe B : Spécification du Pipeline d'Ingestion SIEM & Dictionnaire de Données

B.1 Dictionnaire des champs d'événements de sécurité

Chaque événement d'audit ingéré par la pile ELK est normalisé selon le dictionnaire de données défini dans le Tableau 18.

TABLE 18. Dictionnaire des données de sécurité de l'index Elasticsearch.

Nom du champ	Type Elasticsearch	Source d'origine	Description et Exemple
@timestamp	date	Logstash	Horodatage ISO 8601 UTC de l'événement.
client_ip	ip	NGINX / FastAPI	Adresse IP d'origine du client ou TPP.
http_method	keyword	Requête HTTP	Verbe HTTP utilisé (GET, POST, PUT, DELETE).
request_path	keyword	Requête HTTP	Chemin d'accès relatif à la ressource ciblée.
http_status	integer	Réponse HTTP	Code de statut HTTP retourné (200, 401, 403, 429).
severity	keyword	Threat Monitor	Niveau de sévérité (INFO, WARNING, CRITICAL).
user_id	keyword	JWT Keycloak	Identifiant pseudonymisé du client bancaire.
latency_ms	float	Middleware API	Temps de traitement interne de la requête en ms.

Ce schéma de données structuré garantit l'homogénéité des index Elasticsearch et accélère les requêtes d'agrégation sous Kibana Discover, autorisant une corrélation instantanée par IP source ou par code d'erreur HTTP.

B.2 Modèle d'ingestion Logstash et filtres Grok

Le pipeline d'ingestion Logstash applique les étapes suivantes :

1. **Collecte continue** : Lecture du fichier `security_audit.log` par Filebeat et transmission sécurisée vers Logstash (port 5044).
2. **Extraction structurée par Grok** : Découpage de chaque ligne par le motif standardisé de capture :

```
[{%TIMESTAMP_ISO8601:log_time}] | {%WORD:log_level} | IP: {%IP:client_ip} | METHOD:
{%WORD:http_method} | PATH: {%URIPATH:request_path} | STATUS: {%NUMBER:http_status:int}
| DURATION: {%NUMBER:latency_ms:float}ms
```

3. **Enrichissement et catégorisation** : Attribution automatique du champ `severity` selon le code HTTP et le statut de blocage.
4. **Stockage et rétention** : Indexation journalière dans des index `security-audit-logs-YYYY.MM.DD`. La politique de rétention conserve les index de sécurité pendant 90 jours et les journaux financiers archivés pendant un an avec accès strictement réservé aux analystes autorisés (RBAC Elasticsearch).

Annexe C : Protocole d’Audit Dynamique (DAST) & Matrice de Tests OWASP ZAP

C.1 Plan de test et matrice de couverture par endpoint

Le dispositif d’audit de sécurité s’articule en deux étapes : (1) l’audit de transport frontal évaluant la terminaison TLS 1.3 et le rejet mTLS strict (HTTP 400) au niveau de NGINX sur le port 443 ; (2) l’audit dynamique applicatif DAST par OWASP ZAP (version 2.14.0 exécutée sous conteneur Docker via `zap-api-scan.py` ciblant `http://banking_backend:8000/openapi.json`, durée 272 s). Ce scan applicatif direct permet d’éprouver la robustesse intrinsèque des 10 endpoints bancaires sans masquage par le régulateur de débit frontal, selon la matrice détaillée dans le Tableau 19.

TABLE 19. Matrice des endpoints bancaires et vecteurs d’attaque simulés lors de l’audit DAST.

Endpoint d’API	Méthode	Vecteurs d’attaque simulés par OWASP ZAP
/api/auth/login	POST	Injection SQL, force brute de mot de passe, falsification de jeton.
/api/auth/2fa	POST	Contournement OTP, rejeu de code temporaire expiré.
/api/accounts/{id}	GET	BOLA (changement d’identifiant de compte), injection de paramètres.
/api/payment/transfer	POST	Manipulation de solde, rejeu de transaction, injection de payload XSS.
/api/admin/users	GET/PUT	BFLA (accès non autorisé), élévation de privilèges.
/api/admin/alerts/stream	GET	Injection SSE, contournement de session d’administration.
/api/beneficiary	GET/POST	Pollution de paramètres, injection d’identifiant tiers.
/api/locations	GET	Traversée de répertoires, injection de commandes.
/api/budget	GET/POST	Manipulation de montants, validation de types de données.
/api/financial-hub	GET	Fuite d’informations sensibles, exposition de métadonnées.

Comme l’atteste le Tableau 19, chaque point d’accès a été éprouvé face aux vecteurs d’attaque pertinents selon sa nature fonctionnelle, confirmant l’absence de vulnérabilités critiques exploitables à distance.

C.2 Critères d’acceptation et seuils d’intégration continue (CI/CD)

Pour autoriser la promotion d’un livrable en environnement de production, les critères d’acceptation sécuritaires suivants sont définis :

- **0 alerte de criticité Élevée (High) ou Critique** : Toute découverte de vulnérabilité BOLA, injection ou défaut d’authentification invalide immédiatement le déploiement.
- **0 alerte de criticité Moyenne (Medium)** : Toute défaillance de configuration TLS ou exposition non maîtrisée de ressource requiert une correction obligatoire.

- **Tolérance maximale de 3 alertes de criticité Basse (Low)** : Acceptation conditionnelle de remarques mineures d'optimisation d'en-têtes HTTP de réponse.

C.3 Synthèse du rapport brut OWASP ZAP (Scan Actif d'API)

Le scan actif complet exécuté par la commande officielle `zap-api-scan.py -t http://banking_bac`
`-f openapi -r zap_report.html` a importé la définition OpenAPI/Swagger de l'API, exploré les 10 endpoints bancaires et injecté des vecteurs d'attaque active (injections SQL/NoSQL, tests BOLA/BFLA, manipulation de paramètres et contournements). Le scan a émis 128 requêtes dynamiques et validé un score global satisfaisant (0 High, 0 Medium, 2 Low, 4 Informational). Le rapport complet est archivé dans `documentation/zap_report.html`.

Annexe D : Algorithme Formel et Automate de la Réponse Active

D.1 Algorithme d'évaluation du score et de déclenchement du blocage

L'Algorithme 1 formalise la logique décisionnelle exécutée en continu par le module **ThreatMonitor** lors de l'analyse des flux de sécurité.

Algorithm 1 Évaluation du score dynamique de menace $T_{decay}(x, t)$ et gestion du blocage applicatif

Require: Adresse IP source x , Code de statut HTTP $code$, Seuil critique S , Facteur d'atténuation λ

Ensure: Mise à jour de l'état de sécurité $SecurityState$

```
1:  $t_{current} \leftarrow \text{HorodatageActuel}()$ 
2: if  $x \in \text{SecurityState.WhitelistedIPs}$  then
3:   return ▷ Ignorer les adresses internes de confiance
4: end if
5:  $T_{prev} \leftarrow \text{ObtenirScorePrecedent}(x)$ 
6:  $t_{last} \leftarrow \text{ObtenirDernierHorodatage}(x)$ 
7:  $T_{decayed} \leftarrow T_{prev} \cdot e^{-\lambda(t_{current} - t_{last})}$  ▷ Application de l'atténuation temporelle
8:  $w \leftarrow 0.0$ 
9: if  $code = 401 \vee code = 422$  then
10:    $w \leftarrow 1.0$  ▷ Échec d'authentification / Force brute
11: else if  $code = 403$  then
12:    $w \leftarrow 0.7$  ▷ Accès interdit / Exploration de ressources
13: else if  $code = 429$  then
14:    $w \leftarrow 0.5$  ▷ Dépassement de quota de débit
15: end if
16:  $T_{new}(x) \leftarrow T_{decayed} + w$ 
17:  $\text{SauvegarderScore}(x, T_{new}(x), t_{current})$ 
18: if  $T_{new}(x) \geq S \wedge x \notin \text{SecurityState.BlockedIPs}$  then
19:    $\text{EmettreAlerteSSE}(x, T_{new}(x), \text{"CRITICAL"})$ 
20:    $\text{LancerMinuteurSecurite}(x, \text{duree} = 30\text{s})$ 
21: end if
```

D.2 Machine à états de la gestion d'une adresse IP source

La gestion du cycle de vie sécuritaire d'une adresse IP au sein de **SecurityState** s'appuie sur cinq états distincts :

- **État Sain (Clean)** : L'adresse IP émet des requêtes nominales ($T_{decay}(x, t) < S$). Toutes les requêtes sont acceptées.
- **État Suspect** : L'adresse enregistre des erreurs isolées ($0 < T_{decay}(x, t) < S$). Le score s'incrémente avec décroissance temporelle.
- **État Compte à Rebours (Countdown)** : Le seuil S est franchi ($T_{decay}(x, t) \geq S$). Une alerte SSE est émise et un compte à rebours de 30 secondes s'active sur le tableau de bord d'administration.
- **État Bloqué (Blocked)** : À l'échéance des 30 secondes sans annulation manuelle par l'analyste, l'adresse IP est inscrite dans le registre des adresses bloquées. Toutes ses requêtes sont immédiatement rejetées avec un code HTTP 403 pendant $t_{unblock} = 15$ minutes.
- **État Liste Blanche (Whitelisted)** : Adresses IP d'infrastructure interne (passerelle NGINX, boucle locale) exemptées de tout blocage.

Références

- Campbell, Brian, John Bradley, Nat Sakimura Sakimura et Torsten Lodderstedt. 2020. *OAuth 2.0 Mutual TLS Client Authentication and Certificate-Bound Access Tokens*. Rapport technique. RFC 8705, IETF.
- Commission Européenne. 2015. *Directive sur les services de paiement (DSP2), Directive (UE) 2015/2366*. Rapport technique. Journal officiel de l'Union européenne.
- Hardt, Dick. 2012. *The OAuth 2.0 Authorization Framework*. Rapport technique. RFC 6749, IETF.
- ISO/IEC. 2022. *ISO/IEC 27001 :2022 Information security, cybersecurity and privacy protection — Information security management systems*. Rapport technique. International Organization for Standardization.
- Jones, Michael, John Bradley et Nat Sakimura Sakimura. 2015. *JSON Web Token (JWT)*. Rapport technique. RFC 7519, IETF.
- Kohnfelder, Loren et Praerit Garg. 1999. *The Threats to Our Products : STRIDE Threat Model*. Rapport technique. Microsoft Corporation.
- OpenID Foundation. 2021. *Financial-grade API 1.0 Advanced Security Profile*. Rapport technique. OpenID Foundation Workgroup.
- OWASP Foundation. 2023. *OWASP API Security Top 10 — 2023*. <https://owasp.org/API-Security/>. Consulté le 08 Mai 2026.
- Rose, Scott, Oliver Borchert, Stu Mitchell et Sean Connelly. 2020. *NIST Special Publication 800-207 : Zero Trust Architecture*. Rapport technique. National Institute of Standards et Technology (NIST).